Deggendorf Institute of Technology, Campus Deggendorf

Faculty of Electrical Engineering and Media Technology

Master Electrical Engineering and Information Technology

Master Thesis

Development of a Laboratory Prototype of a Rotational Viscometer

Entwicklung eines Laborprototypen eines Rotationsviskometers

Presented in partial fulfillment of the requirements

for the Degree of:

Master of Science (Electrical Engineering & Information Technology), Deggendorf Institute of Technology, Bayern, Germany

Submitted by: Rahul Kuravanparambil Rajan, 12201084

Supervisor: Prof. Harald Zimmermann

Deggendorf, 20.04.2025

Declaration of Original Authorship

I hereby declare that this final thesis is my own work, has not been submitted for any other degree at any other university or institution, does not contain or use any sources or resources other than those referenced, and that all direct and paraphrased quotes have been duly cited as such.

Deggendorf, 20/04/2025

I hereby consent to my completed final thesis being made available to a broader public through the library of Deggendorf Institute of Technology.

Consent: Yes ☐ No ☒

I certify and warrant that I am the sole owner and holder of all rights to the final thesis, including the right to use the tables, figures, illustrations, plans, or similar, submitted therein, and that no third-party rights or claims, nor any statutory provisions, will be infringed as a result of its publication.

Deggendorf, 20/04/2025

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Prof. Harald Zimmermann, for his exceptional guidance and support throughout this research. His expertise and valuable insights were instrumental in overcoming the challenges of this study and achieving meaningful results.

I would also like to extend my thanks to the technical supervisors, Roland Stange and Florian Reischl, whose practical assistance and technical support were essential in carrying out the experimental aspects of this research. His dedication and expertise played a significant role in the successful completion of this project.

I am deeply grateful to all TAZ Spiegelau staff, my family, and friends for their unwavering encouragement and support throughout this journey. Lastly, I extend my appreciation to my colleagues and the academic community, whose inspiration and motivation fueled my curiosity and commitment to this field. This research would not have been possible without the guidance, resources, and support of everyone who contributed along the way. Thank you for making this endeavour both achievable and rewarding.

Abstract

This project has developed a laboratory rotational viscometer prototype used for determining the viscosity of liquids with relevance to industrial glass applications. The device construction follows a basic model toward creating future implementations of real-time in-line viscosity monitors. The setup enables viscosity measurement of glass-forming melts under controlled thermal and mechanical conditions because viscosity determines flow behaviour and shows strong temperature dependence.

The system uses a torque measurement method through a stepper motor-driven ceramic rod spinning inside test fluids using dynamic rotary torque sensors. A Raspberry Pi system allows motor control and data logging, and real-time interface monitoring through a specialised graphical user interface. The setup was tested with silicone oils, which have viscosities of 5000 cSt, 10000 cSt, 20000 cSt, 30000 cSt, 40000 cSt, and 60000 cSt under different RPM levels and immersion depths. The strong temperature effects on viscosity led to the use of a thermometer for fluid temperature observation according to established research findings. The viscometer used a design that has an infinite cup geometry assumption to achieve consistency and physical accuracy. The calculation of viscosity runs independently of changes in radial distance between the rotating rod and container wall due to this model's unique configuration. This achieves better results in comparison to standard coaxial cylinder technologies. The simplified analytical system works independently of vessel diameter when the infinite cup condition remains near accurate.

Experimental tests on the system were done across different levels of immersion depth combined with various rotational speed values to demonstrate its operational reliability in dynamic conditions. The obtained torque values, together with angular velocity measurements, allowed the computation of viscosity results while using geometric factors calculated from infinite cup model principles. The main engineering obstacle involved reducing mechanical errors, particularly involving poor alignment between the rotating shaft and the sensor axis, which strongly impacts torque measurements. The alignment precision, along with rigid mechanical structure integration, took precedence because literature shows that mechanical tolerances negatively affect the viscometric accuracy.

This device allows a reliable and cost-efficient technique for viscosity measurement through its modular structure. Future adoption into commercial glass processing flowlines is achievable because this system creates standards for monitoring viscosity continuously, which optimizes product consistency and energy efficiency and quality outcomes.

Key words: Viscosity determination, Glass-forming melts, Thermal and mechanical control, Torque measurement, Stepper motor, Ceramic rod, Dynamic rotary torque sensors, Raspberry Pi control, Data logging, Graphical interface monitoring, Immersion depths, Mechanical tolerances, Product consistency.

Contents

Acknowledgement ..... **iii**
Abstract ..... **iv**
Contents **1**
List of Figures **1**
List of Tables **2**
Chapter 1: Introduction **1**
1.1 Overview 1
1.2 Problem Statement 2
1.3 Objectives 2
1.4 Glass Viscosity 3
1.5 Basic Equations of Viscosity 5
Chapter 2: Literature Background **7**
2.1 Glass 7
2.2 Viscometer 9
Chapter 3: Methodology **13**
3.1 Design and Development 13
3.2 Schematic Diagram 15
..... 15
3.3 Components and Devices 16
3.4 Integration of Stepper Motor to Raspberry Pi through Driver 29
3.5 Integration of Torque Sensor to Raspberry Pi 5 31
3.6 Graphical User Interface (GUI) for Real-Time Control and Monitoring 35
3.7 Data Acquisition 37
3.8 Workflow of the system 39
Chapter 4: Testing and Calibration **40**
4.1 Experimental setup 40
4.2 Trial run without sample 41
4.3 Testing with a sample 41
4.4 Calibration 43
4.5 Final Calibration- Data Processing and Polynomial Correction 44
4.6 Python Implementation 46
Chapter 5: Results **47**
5.1 Result of Final Testing 47

5.2 Example of final raw sensor data 48
5.3 Error Analysis 49
Chapter 6: Conclusion and Future Scope 50
6.1 Findings 50
6.2 Future scope 51
Bibliography 1
Appendix: 1
6.3 Results and other documents 1
6.4 Python code 1

List of Figures

Figure 1.1 Imaginary moving layers in a flowing material 5
Figure 2.1 Illustration of relaxation mechanisms in a glass system. 7
Figure 2.2 Spectrum of relaxation of glass-forming liquids, compared with dynamic spectra of crystals 8
Figure 2.3 Temperature dependence of volume and enthalpy during liquid-to-glass transition. 8
Figure 2.4 Schematic representation of glass manufacturing process 9
Figure 3.1 Schematic view of the whole setup 15
Figure 3.2 Mechanical setup with motor and sensor 16
Figure 3.3 Stepper motor holder 17
Figure 3.4 Coupler for connecting between motor shaft and the sensor 17
Figure 3.5 Cylindrical Rod 18
Figure 3.6 Stepper Motor 18
Figure 3.7 schematic of stepper motor 19
Figure 3.8 Specifications of the controller 20
Figure 3.9 Block diagram of the controller 21
Figure 3.10 Torque sensor ZHKY2050 21
Figure 3.11 Multimeter 22
Figure 3.12 Thermometer 23
Figure 3.13 Switching-mode power supply 24
Figure 3.14 Raspberry Pi 5 25
Figure 3.15 The interface of Thonny IDE on Raspberry Pi IOS 28
Figure 3.16 The interface of the PyCharm IDE 28
Figure 3.17 Data sheet of ZHKY2050 torque sensor for calibration 31
Figure 3.18 push-pull output signal of zhky2050 32
Figure 3.19 Electrical wiring and output signal connection of ZHKY2050 33
Figure 3.20 The signal conditioning circuit with voltage divide and RC Band filter 34
Figure 3.21 Graphical User Interface 35
Figure 3.22 The folder view shows stored Excel data files and corresponding plot images. 37
Figure 3.23 Real-Time Viscosity vs Time plot from GUI 38
Figure 3.24 Flow chart of the entire system 39
Figure 4.1 Mechanical setup with a sample for a trial run 40
Figure 4.2 plot of the result in Table 7 42
Figure 4.3 Testing sample (SILICON oil 58.2 Pa.s) 43
Figure 4.4 calibration curve of the first calibration 44
Figure 4.5 this figure illustrates the plot of the calibration factor versus torque 45
Figure 5.1 Comparison between measured and required viscosity 48
Figure 5.2 Torque over time (29.1 Pa.s Silicon Oil) 49

List of Tables

Table 1: Dimensions of Figure 7 (mm) 19
Table 2 Minilog control commands of the motor controller 29
Table 3 Details regarding circuit connection 34
Table 4 Details of each section in the control panel of the GUI 36
Table 5 Example of data collected in a single test 38
Table 6 Details of samples used for testing and calibration 41
Table 7 Test results of silicon oil in two different viscosities 42
Table 8 Result after the first calibration 44
Table 9 Final Results of Testing with silicon oil in different viscosities 47
Table 10 Final raw sensor data (29.1 Pa.s Silicon Oil) 48
Table 11 Mean and Standard Deviation for Torque and Viscosity 49

Abbreviation

DIT	Deggendorf Institute of Technology
TAZ	Technologie Anwender Zentrum
GUI	Graphical User Interface
RPM	Revolutions Per Minute
cSt	Centistokes
VFT	Vogel-Fulcher-Tammann
MEMS	Micro-Electro-Mechanical Systems
PWM	Pulse Width Modulation
RS485	Recommended Standard 485 (serial communication standard)
USB	Universal Serial Bus
HMI	Human Machine Interface
PLC	Programmable Logic Controller
IP	Ingress Protection (rating for enclosures)
DC	Direct Current
JSON	JavaScript Object Notation
PID	Proportional-Integral-Derivative (controller)
PCA	Principal Component Analysis
CW	Clockwise
CCW	Counter-Clockwise

Chapter 1: Introduction

1.1 Overview

All primary stages of glass production depend on the fundamental material parameter known as viscosity, which guides glass melting along with refining and mould formation, and annealing steps[1]. The flow behaviour of molten glass emerges from viscosity, which affects product quality control and creates system stability requirements as well as operational energy expenditure. Viscosity functions as a critical factor in glass manufacturing, yet its in-line measurements remain difficult to implement at production facilities because of high temperatures combined with glass melt corrosiveness[2].

Industrial operators measure viscosity by tracking temperature changes since they maintain fixed relationships between the two variables. Manufacturers use conventional methods that depend on the Vogel-Fulcher-Tammann (VFT) model, along with empirical viscosity-temperature relationships, to calculate viscosity under set thermal conditions [3]. The easy assumption about viscosity ignores the multiple interacting elements that influence the process[4], including the composition of the mixture and the uncertain nature of recycled components and batch variations[5].

Research has proven that silicate and non-silicate melts show non-linear viscosity behaviours that depend on composition, while temperature alone fails to predict these effects. The practice of managing glass quality through temperature controls alone creates potential risks regarding product standards, along with operational efficiency.

None of the current viscometric systems fulfils the fundamental requirement of becoming reliable and real-time measurement tools within industrial glass furnaces. The measurement capabilities and durability requirements for high-temperature industrial processes surpass the functionality of conventional benchtop and MEMS-based viscometers, even in laboratory settings[6][7]. Industrial glass production demands appropriate in-line viscometric solutions because existing technology currently does not meet operational requirements.

The industry-focused Technologie Anwender Zentrum (TAZ) Spiegelau within Technische Hochschule Deggendorf recognised the industry requirement for strong viscosity measurement solutions that could operate in-line. The research investigates design principles and methods that will aid future commercial implementations.

[1] (Krishnakumar)
[2] (J. C. Mauro)
[3] (Zanotto)
[4] (Nascimento)
[5] (Stillinger)
[6] (Singh)
[7] (Shamuratov)

1.2 Problem Statement

The existing industrial viscosity monitoring system faces issues because it uses temperature as a substitute for direct viscosity measurement. Modern glass manufacturers evaluate glass melt quality by maintaining target temperature thresholds since they believe that equivalent temperatures provide equal viscosity and quality characteristics[8] [9]. Glass composition, together with impurities and recycled content, and volatile species, causes significant viscosity variations at equivalent temperatures, while this testing approach ignores these factors[10][11][12].

Manufacturers cannot detect viscosity deviations until they result in quality issues, so they end up producing defective products and wasting energy while their processes operate less efficiently. The absence of real-time viscosity monitoring stops manufacturers from implementing corrective actions mechanically or manually during processing time.

The main difficulty exists in obtaining a durable in-line viscometer that functions properly in typical glass manufacturing furnace environments. Industrial glass plants face operational challenges with commercial viscometers that were developed for benchtop or MEMS-scale purposes, since these tools cannot survive in the continuous, high-temperature, corrosive environments[13] [14]. Literature documents this issue universally as a major technological hindrance to achieving process optimisation and quality control goals.

The present work presents a rotational viscometer prototype for laboratory testing with future industrial scalability as its main objective. The device prototype represents a step forward because it enables the transition from temperature-based indirect measurements to direct viscosity measurements for instant composition-dependent process control that upgrades manufacturing capabilities[15].

1.3 Objectives

- **Prototype Development:** To develop a prototype laboratory viscometer for accurate measurement of the dynamic viscosity of molten glass-like fluids.
- **Method Validation:** To validate torque-based viscosity measurements with silicone oils operated at controlled RPM and depths at ambient temperatures to deliver accurate viscosity readings inside the operational bounds of the prototype.
- **Data Acquisition System:** To develop a Raspberry Pi system that functions as a data acquisition interface for monitoring process parameters in real-time and permanent data storage.
- **Industrial Relevance:** The model serves as the base to build an advanced viscometer system that can withstand glass furnace environmental conditions.
- **Quality Improvement:** The system needs development to measure viscosity directly in glass manufacturing and replace temperature-based quality assessment methods, which ensure better manufacturing control and waste minimisation.

[8] (Krishnakumar)
[9] (J. C. Mauro)
[10] (Zanotto)
[11] (Ojovan)
[12] (Stillinger)
[13] (Nascimento)
[14] (Singh)
[15] (Shamuratov)

1.4 Glass Viscosity

Glass-forming liquids need viscosity modelling to gain full comprehension of their behavioural characteristics throughout different temperature ranges, including the supercooled and transition states. The strong temperature sensitivity of viscosity exceeds the predictive capacity of basic Arrhenius expressions in these materials. Three different three-parameter models exist for empirical and semi-empirical modelling of viscosity data since they provide various levels of physical interpretation alongside varying accuracy across wide temperature spans.

1.4.1 Mathematical Models of the Viscosity Equation

This model, which describes viscosity variations in glass-forming systems, was first developed by Gordon S. Fulcher in 1925 and remains widespread for use today. The equation described as Fulcher's Equation serves as the model framework

Gardon Fulcher's equation

Equation 1

$$\log \eta(T) = A + \frac{B}{T - T_0}$$

The developed model efficiently predicted viscosity across more than ten orders of magnitude through three adjustable parameters: A, B, and T0. Fulcher obtained the mathematical expression by studying laboratory data of sodium silicate glasses containing Mgo, Cao, Al_2O_3, and B_2O_3. The work by Fulcher proved the equation to be an essential technical tool for glass science during its time of publication. Independent initial development of the equation led to its discovery of congruence with expressions from Vogel (1921) and Tammann (1926). The model received its recognized name as Vogel–Fulcher–Tammann (VFT) in the broader scientific literature after its independent origin[16][17].

Vogel-Fulcher-Tammann (VFT) equation

EQUATION 2

$$log_{10}\, \eta(T, x) = log_{10}\, \eta\infty(x) + \frac{A(x)}{T - To(x)}$$

The widely recognized equation for temperature effects on viscosity retains the name Vogel-Fulcher-Tammann (VFT) equation even though Wilhelm Vogel (1921) and Gustav Tammann developed it separately. Vogel presented the initial formulation of the equation for supercooled liquids before Tammann applied it to glass transition research. These three researchers collectively demonstrated

[16] (J. C. Mauro)
[17] (Shamuratov)

their understanding of glass-forming material viscosity, which resulted in the combined name Vogel-Fulcher-Tammann being applied to this equation.

The VFT equation finds its value in describing non-Arrhenius behavior patterns in glass-forming liquids. The VFT equation deviates from Arrhenius-type models because it contains a temperature T0 where viscosity would theoretically become infinite. The VFT equation demonstrates strong efficacy when examining the viscosity surge that occurs when glass-forming liquids reach the glass transition temperature.

The VFT equation is used extensively in glass manufacturing, polymer science, and material processing because accurate viscosity predictions enable the control of production parameters. Experimental tests determine the empirical values of A, B, and T0 for various glass compositions, enabling the equation to be used with different glass materials. The VFT equation has proven essential for studying glass and supercooled liquids because it describes viscosity behaviors over various magnitude ranges (J. C. Mauro).

The Avramov–Milchev (AM) Model

The Avramov–Milchev (AM) model functions as a common three-parameter description method for glass-forming liquid viscosity evaluation. The creators derived this model from atomic hopping theory as they sought to explain basic structural changes that occur in supercooled liquids. The AM equation is presented below.

EQUATION 3

$$\boldsymbol{log\,\eta\,(T) = log\,\eta_{\infty} + \frac{\beta}{T^{\alpha}}}$$

- $\eta(T)$ is the viscosity at temperature T,
- η_{∞} is the high-temperature viscosity limit,
- β and $T^{\alpha}$are fitting parameters that describe the non-Arrhenius behaviour of viscosity.

Unlike the Vogel–Fulcher–Tammann (VFT) equation, the Arrhenius Model does not create such an artificial singularity at the temperature T0. The model uses atomic-scale mechanics to describe viscous flow physically while linking viscosity to the probability of atomic energy barrier jumps.

The AM equation shows outstanding fitting capabilities for viscosity data across extensive temperature ranges, which makes it appropriate for each glass-forming inorganic and organic liquid. The model demonstrates consistent measurement errors while estimating viscosity values at low temperatures because it shows lower-than-expected viscosity results. High-temperature operation in the AM model results in unphysical results, which stem from entropic divergence[18].

From a practical standpoint, the AM model continues to serve researchers who need to analyse experimental data yet has certain boundaries when used for a theoretical understanding of supercooled liquids. When used alongside VFT models and entropy-based approaches, the AM model plays an important role in enhancing glass transition research.

[18] (J. C. Mauro)

1.5 Basic Equations of Viscosity

Viscosity is the resistance of a fluid (liquid or gas) to a change in shape or movement of neighbouring portions relative to one another. Viscosity denotes opposition to flow. Viscosity is one of the most important properties of a flowing material. E.g., in the glass industry, viscosity determines melting conditions, working and annealing temperatures, the rate of refining, maximum use temperature, and crystallisation rate. On Earth, the behaviour of magma and, hence, volcanic eruptions and lava flow rates depend directly on the viscosity of the mantle. Viscosity is a property of fundamental importance both for the molten and solid states of amorphous materials. Fluids can be categorised into two types; firstly, there are Newtonian fluids, which possess low molecular weights, and their viscosity is independent of time and shear rate, and thus depends only on temperature, pressure, and molecular properties. Secondly, non-Newtonian fluids are of high molecular weight, possessing a variable viscosity depending on the strain rate

Maxwell's kinetic theory

The viscosity quantifies the resistance of the material to flow and indicates the ability to dissipate momentum. The momentum balance of a Newtonian fluid is described at the macroscopic level by the Navier–Stokes equations. At the microscopic level, the viscosity arises because of a transfer of momentum between fluid layers moving at different velocities, as explained in Maxwell's kinetic theory (Ojovan)(1).

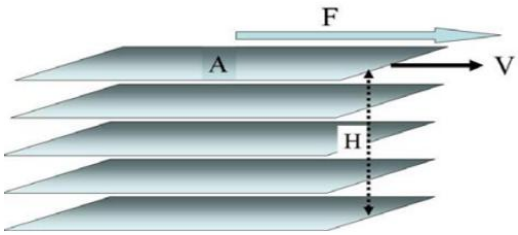


FIGURE 1.1 IMAGINARY MOVING LAYERS IN A FLOWING MATERIAL

EQUATION 4

$$\eta = \frac{(f\ /\ A)}{(v\ /\ H)}$$

where f is the force applied to the surface of the area A, v is the velocity, and H is the separation height between the two plates. In terms of imaginary layers, the tighter the bound layers, the more difficult their motion and the higher the resulting viscosity[19][20].

[19] (Zanotto)
[20] (Krishnakumar)

Newton's Law of Viscosity

Newton's law of viscosity describes the relationship between the shear stress and the velocity gradient in a **Newtonian fluid**. It states that the shear stress acting on a fluid layer is directly proportional to the rate of change of ($\frac{du}{dy}$) velocity across the fluid layer:

EQUATION 5

$$f = \mu A \frac{du}{dy}$$

Where:

- F is the applied force
- μ is the dynamic viscosity
- A is the area of the surface in contact with fluid
- $\frac{du}{dy}$ is the velocity gradient

EQUATION 6

$$\tau = \mu \frac{\partial u}{\partial y}$$

This equation shows that a higher viscosity or a greater surface area A increases the force required to maintain the fluid's motion.

The mentioned ratio u/y is the *rate of shear deformation* or shear velocity. It is the derivative of the fluid speed in the direction parallel to the normal vector of the plates. If the velocity does not vary linearly with y, then the appropriate generalisation is:

Chapter 2: Literature Background

2.1 Glass

Archaeological findings demonstrate that Egyptians, alongside the people of Eastern Mesopotamia, produced the earliest manufactured glass approximately 5000 years ago during 3000 BC. The Stone Age civilisation used volcanic glass obsidian to make weapons, together with decorative objects. Archaeological evidence shows that the first creation of hollow glass objects started during the 16th century BC in Mesopotamia, and parallel discoveries were made simultaneously in Mycenae and China, and Egypt. A formalised method of glass synthesis has existed since the time of the earliest known glassmaking manual written by King Ashurbanipal of Assyria from approximately 650 BC. Glassblowing became a revolutionary breakthrough in the 1st century BC because it transformed the industry by enabling the mass production of glass items[21].

Transparency in glass became evident for architectural applications during the Roman era after scientists found it in Alexandria. Transparent glass dramatically increased its European economic value, together with its aesthetic worth throughout the continent after its discovery. During the 17th century, the invention of lead glass by George Ravenscroft transformed how scientists enhanced optical characteristics. The scientific investigation of glass made significant progress during the 18th

and the 19th century, in addition to the previous increase in artisanal glass practice. More than 120,000 glass-related scientific publications appeared following the mid-twentieth century, according to Mauro et al[22]. Traditional descriptions of glass materials show these solids to be brittle, transparent materials with hard qualities. Glasses exhibit clear differences from crystalline solids because they lack extended structural organisation. The matter state of glasses rests between stability and presents as an amorphous form, which scientists term a supercooled liquid with exceptionally high viscosity. Their behaviour near the glass transition temperature T_g is especially complex and is characterised by dramatic changes in physical properties over narrow temperature intervals.The relationship between external stimuli, along with stress and electric fields and magnetic fields, and temperature with the glass matrix appears in Figure 2.1. Strain alongside dielectric displacement and enthalpy changes alongside magnetic outputs constitute the system's response based on the input nature. The material's mechanical and thermal behaviour depends primarily on this relaxation mechanism[23].

FIGURE 2.1 ILLUSTRATION OF RELAXATION MECHANISMS IN A GLASS SYSTEM.

[21] (Basumatary)
[22] (Glass)
[23] (source1)

The representation of relaxation spectra displays more detail in Figure 2.2. Multiple relaxation zones spanning from the α-relaxation to β-relaxation and fast processes and boson peak appear in the upper segment of the diagram to illustrate glass-forming liquid dynamics at various scales. Structures of crystalline solids demonstrate low-frequency phonon peaks since they present the basic arrangements and motions of crystalline solids. The diagrams show the intricate nature of glass relaxation phenomena because glasses contain no periodic atomic structure yet respond across a wide range of frequencies[24].

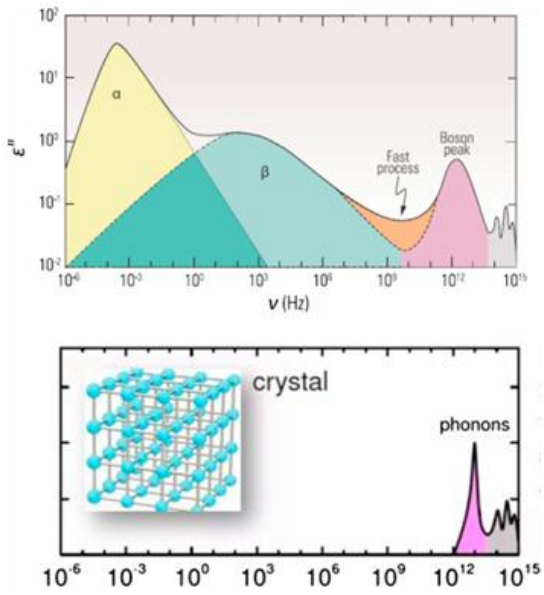


FIGURE 2.2 SPECTRUM OF RELAXATION OF GLASS-FORMING LIQUIDS, COMPARED WITH DYNAMIC SPECTRA OF CRYSTALS

A supercooled liquid transition to glass occurs naturally without specific phase changes by showing extended increases in viscosity. Figure 2.3 shows the changes in volume and enthalpy that occur when temperature increases. A cooling process that maintains the material in its supercooled condition below its melting point will quickly transform it into glass at (tg). The freezing behaviour of glass differs from crystalline materials because it demonstrates sharp phase changes at its freezing point.

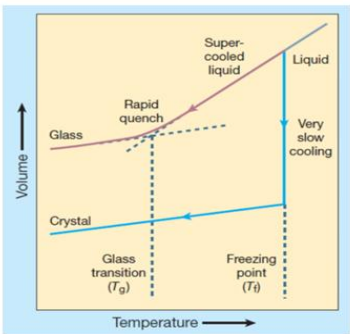


FIGURE 2.3 TEMPERATURE DEPENDENCE OF VOLUME AND ENTHALPY DURING LIQUID-TO-GLASS TRANSITION.

[24] (source1)

Glasses exhibit their structural behaviour through both short-range and medium-range order, which leads to bridging and non-bridging oxygen species formed through the network formers and modifiers. The physical attributes of density, refractometry, thermal expansion coefficients, and viscosity values react to both material chemical makeup and processing temperature changes. The correct control of glass manufacturing processes relies on accurate viscosity measurement because, without it, researchers cannot fully understand flow dynamics [25] . The macroscopic behaviour of materials in glasses depends on two dynamic relaxations, which include α-relaxation for cooperative molecular motions throughout the substance and β-relaxation signals localized movements. Both the rigidity and relaxation timescales within the structural framework emerge from chemical makeup and rate of cooling[26]. Research into glass and its properties will continue because it displays dual liquid-solid behaviour, together with intricate relaxation characteristics and longstanding human use. Several measurements presented in this work help establish more precise connections between glass structure and its observed properties.

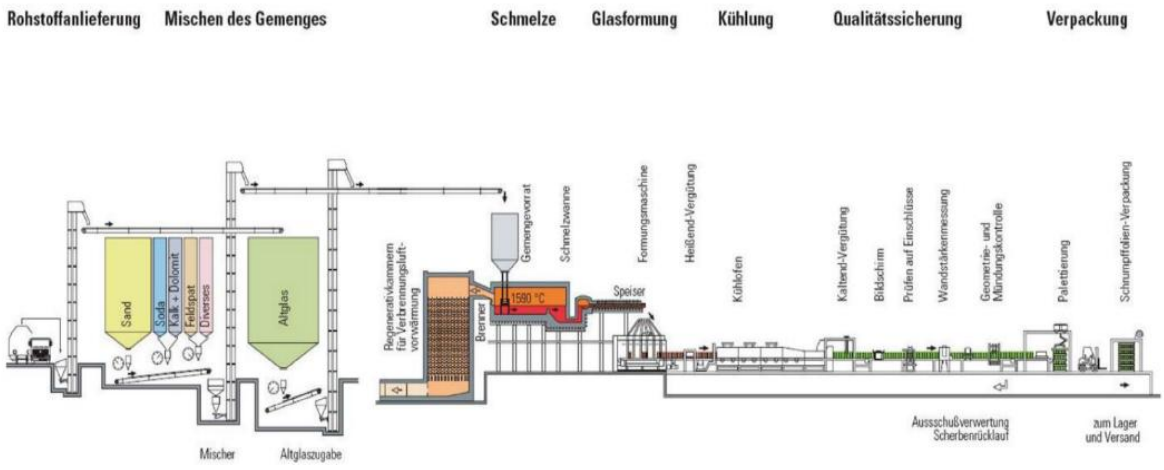


FIGURE 2.4 SCHEMATIC REPRESENTATION OF GLASS MANUFACTURING PROCESS

2.2 Viscometer

Viscosity stands as one of the fundamental rheological characteristics that require exact measurement through multiple industrial and scientific applications. A fluid demonstrates viscosity because it opposes both flow motion and shear deformation. The real-time accurate measurement of viscosity serves as a vital tool for process optimization and quality control, as well as material characterization, especially in glass manufacturing, together with biomedicine and food processing, and petrochemical industries[27].

Rotational viscometers have showcased extensive use because they both demonstrate high stability and provide viscosity measurement across different ranges under regulated shear conditions. These devices function by measuring the necessary torque needed to turn their spindle or bob as they enter a solution where they excel at evaluating non-Newtonian fluids. The requirement for fluid immersion, along with big sample amounts, restricts the portability and sensitivity of these devices, particularly at

[25] (Affatigato)
[26] (source1)
[27] (Singh)

high temperatures and in reactive settings[28] [29]. Multiple proposed designs aim to solve the ongoing problems with current viscometer systems. New sensor technologies combined with micro-electromechanical systems (MEMS) and non-contact measurement principles enabled researchers to build small yet precise viscometers that serve specific applications.

2.2.1 Types of Viscometers

MEMS-based Viscometers

Modern MEMS technologies enabled the development of compact viscometers, which provide superior sensitivity alongside quick responses during the testing of low-volume samples. MEMS-based devices employ multiple operating principles to measure fluid viscosity, including pressure sensors, along with acoustic detection and cantilever beam resonance, and microfluidic chips. MEMS-based viscometers successfully operate within biomedical, inkjet, and oil and food industries while providing the benefits of a disposal option and self-calibration qualities, and compatibility for Newtonian and non-Newtonian fluid testing[30]. The operation of cantilever-based MEMS viscometers depends on changes in resonance frequency and damping ratio that occur when a beam oscillates in a fluid medium. The operational capabilities of diaphragm-based sensors and vibrating wire viscometers rely on stress-strain relationships and frequency shifts to determine fluid properties.

Penetration and Penetroviscometers

The penetration-type viscometer analyses the resistance force the probe or inner cylinder encounters while moving through the fluid. Viscometers have received modifications that make them acceptable for steady and transient shear property measurements. Bikerman developed the penetroviscometer for measuring shear deformations at high speeds, commonly observed when processing shear-thickening suspensions[31]. It was validated mathematically for this application. The instrument provides precise instant viscosity measurements in non-steady conditions while eliminating the common rheometer limitations from both the instrument and fluid inertia.

Thread and String Viscometers

The mechanical response of thread or fibre suspension in viscous media allows thread-based viscometers to determine shear-produced displacements and damping effects. The vibrating string method adopts the principles of resonance to measure fluid viscosity. The devices function well for viscosity evaluations conducted without containers or performed under harsh temperature conditions, especially featuring molten glass or metallic systems[32].

[28] (Shamuratov)
[29] (Singh)
[30] (Singh)
[31] (Fakhari)
[32] (Basumatary)

Levitating Sphere Viscometers

A levitating sphere viscometer uses a rotational mode to drive the magnetic suspension of spheres through fluid. To establish fluid viscosity, we measure torque together with rotating angular speed. These measurement systems function excellently for sterile and toxic or corrosive fluids because they depend only on the rotating body in direct contact with the sample. High measurement ranges can be achieved at high viscosities because frictional losses are eliminated, and actuation current control is precise[33] .

Fibre Optic Sensor-Based Viscometers

The delay time of light transmission through cladless fibre areas serves as an indicator to measure fluid viscosity in optical sensor-based systems. The delay, referred to as transition time difference (TTD), displays a correlation to fluid velocity and, therefore, tracks viscosity levels. Devices utilizing such technology provide non-contact viscosity assessment along with possibilities to perform automated and remote industrial diagnostic procedures.

Online and Vibrational Viscometers

Recent research development has focused on creating a linear drive system-based online viscometer, which stands out as an innovative approach for real-time high-temperature viscosity measurement. The concept demonstrated its preliminary use in glass manufacturing for maintaining consistent viscosity, which supports accurate dimension control and quality production. A configuration of this method sees researchers submerge a ceramic rod into a molten fluid while subjecting it to damped forced vibration. The electromagnetic drive system actuates the rod to perform linear motion, while the recorded vibration data helps determine fluid-caused viscous damping. Real-time fluid viscosity measurement relies on the analysis of damping behavior collected from the system. The relationship between vibrational amplitude and damping ratio, and fluid resistance enables non-intrusive continuous viscosity monitoring according to this promising method [34].

Krishnakumar constructed the system with a ceramic rod combined with a linear actuator alongside a PLC-based TwinCAT control system that integrated the feedback loop according to a recent master's thesis. Laboratory experiments controlled the rod's displacement and damping measurements through testing it with reference fluids such as water, glycerin, and sugar syrup to reproduce glass melting behaviour. Experimental trials showed that vibration-damping techniques could effectively track viscosity changes when temperature levels were considered moderate. The measurement stability and accuracy heavily depend on three key variables: mechanical alignment, rod material selection, and immersion depth. The research suggested future integration of this method into industrial channels for glass conditioning through viscosity control systems, despite current limitations regarding high-temperature testing capacity.

[33] (Clara)
[34] (Shamuratov)

Rotational Viscometers

Lab and industrial entities widely use rotational viscometers because these instruments measure fluid viscosities across a wide spectrum under defined shear conditions. The measurement of torque needed to turn a spindle or bob, or rotor, which is submerged in fluid, enables the viscosity determination based on flow resistance data. The measurement methods rely on concentric cylinders and cone-and-plate setups, and parallel plates because they function properly with fluid types ranging from Newtonian to non-Newtonian. The instruments demonstrate reduced effectiveness when operating at extreme temperatures, together with chemically harsh conditions, which are present during molten glass processing. The system incorporates a stepper motor-controlled ceramic rod inside fluid solutions and measures the system torque with a rotary torque sensing device. The present prototype handles viscosity measurements through an open setup built with an infinite cup approximation, thus reducing its requirement for specific radial gap alignment between the rod and vessel. The device features improved design capabilities for depth adjustments that provide a simplified modelling process.

Chapter 3: Methodology

3.1 Design and Development

The working of a Rotational viscometer is according to the load generated on the rotating Rod in the viscous fluid. The Torque generated on the rod is then used to find the viscosity of the glass fluid.

3.1.1 Working Principle

The rotational viscometer operates on the principle of measuring the resistance of a fluid to rotational shear. During the measurement, a rotating rod is immersed in the sample fluid and rotated at a controlled speed. The torque required to maintain this rotation is directly proportional to the viscosity of the fluid.

The viscometer consists of a motor-driven spindle, which is submerged in the test sample contained in a specialised chamber. As the spindle rotates, the fluid exerts a resistive force against its motion. A torque sensor detects this resistance and converts it into a viscosity reading. The measurement process is controlled through a precise speed regulation mechanism, ensuring that the spindle rotates at a constant velocity.

The resulting viscosity value is calculated based on the applied torque, rotational speed, and the geometric factors of the spindle and sample chamber. This method allows for accurate and repeatable viscosity measurements across a range of fluid types.

Equation for the measurement of viscosity

EQUATION 7

$$\mu = \frac{\boldsymbol{T}}{\boldsymbol{\pi} * \boldsymbol{d}^2 * \boldsymbol{h} * \frac{\boldsymbol{\omega}}{\boldsymbol{2}}}$$

Angular Velocity (ω): The rate at which the spindle rotates, calculated using:

EQUATION 8

$$\omega = \frac{2 * \pi * RPM}{60}$$

where μ is the Viscosity measured and immersion depth h is the depth to which the spindle is submerged in the fluid. Affects the torque measurement and must be kept constant for accurate readings. Rod Radius r is the radius of the rotating spindle. It influences the shear stress exerted by the fluid. Rod Diameter d defined as, and it is used in viscosity calculations to relate shear stress and strain rate.

EQUATION 9

$$T = 0.1 * f - 1$$

Where T is the torque required to measure corresponding Viscosity, and the torque sensor has a frequency f. 0.1 and 1 are the offset values for finding equivalent Torque from frequency.

To make viscosity independent of RPM, we multiply the measured torque by, modifying the equation as:

EQUATION 10

$$\mu = \frac{\boldsymbol{T}}{\boldsymbol{\pi} * \boldsymbol{d}^2 * \boldsymbol{h} * \frac{\boldsymbol{\omega}}{\boldsymbol{2}}}$$

This adjustment ensures that the viscosity measurement reflects the true fluid properties regardless of the spindle's rotational speed.

Since the container or space holding the test fluid can vary in size across different industries, the system is typically modelled as having an infinite cup. This assumption ensures that variations in container dimensions do not influence viscosity measurements, making the method consistent across applications.

3.2 Schematic Diagram

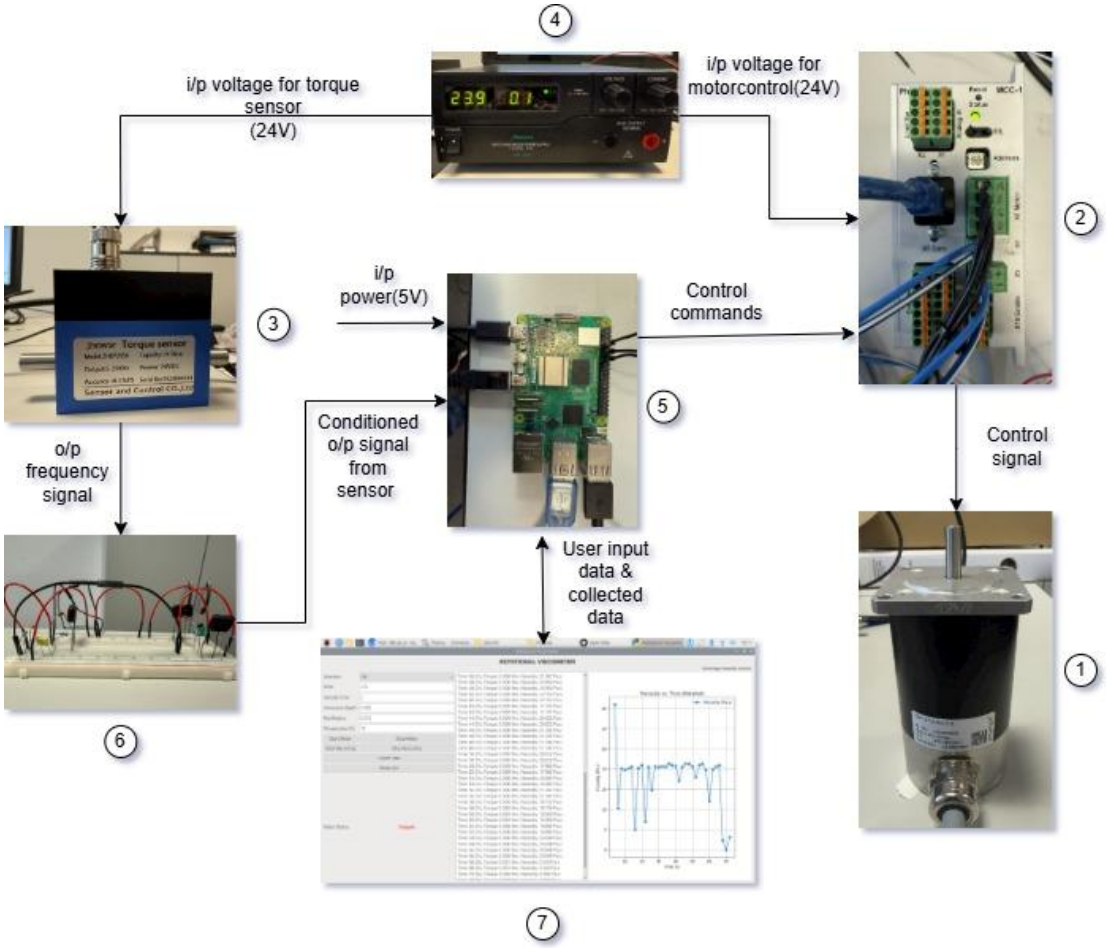


FIGURE 3.1 SCHEMATIC VIEW OF THE WHOLE SETUP

1. Stepper motor
2. Stepper Motor Controller MCC-1-32-48-USB-H
3. Dynamic Torque Sensor
4. Switching Mode Power Supply
5. Raspberry Pi 5
6. Signal conditioning circuit
7. Graphical User Interface

This schematic demonstrates the hardware configuration that integrates all components of the laboratory prototype rotational viscometer. The schematic presents the control signal movement and sensor feedback operations between core elements, including the stepper motor, torque sensor, motor controller, Raspberry Pi 5, and visualization module.

3.3 Components and Devices

3.3.1 Mechanical parts

The stepper motor received its mounting support through a customized holder produced from Fusion 360 designs that facilitated precise relationships between all mechanical parts. The design enabled the holder to bear the motor weight while minimizing vibrations and offering slight modifications for shaft alignment. After structural verification of the finalized design, we obtained an appropriate off-the-shelf bracket through Amazon purchasing. The STEPPERONLINE bracket, designed for NEMA 23 stepper motors, served as the selected part. The metal bracket uses black plastic spray coating and features multiple mounting holes, together with elongated base slots. The bracket included features that enabled easy motor positioning during setup and allowed users to align the motor exactly coaxially with the torque sensor.

FIGURE 3.2 MECHANICAL SETUP WITH MOTOR AND SENSOR

Holder

The motor integration occurred by attaching it to the bracket using its flange, followed by bracket fixation onto the base platform of the system. The precise lateral alignment needed for torque transfer was obtained through the elongated slots on the bracket base.

FIGURE 3.3 STEPPER MOTOR HOLDER

The bracket served to both stabilize and readjust the motor, which in turn reduced movement between components and absorbed external tremors that might impact sensor data.

Shaft coupler

The motor shaft required connection to the torque sensor through a flexible shaft coupler. The mechanical buffer made of aluminum and polyurethane serves as a compensating device that sustains minor angular and radial misalignment between parts. Through its design, the coupler functions to stop the passage of mechanical shocks and vibrations that could occur between parts. The specifications indicate this product's ability to transmit high torque through elastic components, which makes it suitable for CNC and industrial applications. A 6.35 mm to 8 mm coupler functions as the chosen component because it fulfills both motor shaft needs and sensor shaft requirements.

FIGURE 3.4 COUPLER FOR CONNECTING BETWEEN MOTOR SHAFT AND THE SENSOR

The transmission line continued with a second shaft coupler to connect the torque sensor with the vertical rod which sunk into the viscous fluid. The second coupler received selection for an 8 mm to 22 mm bore size because it needed to fit the torque sensor shaft's diameter against the rod's dimension. Two benefits arise from the bigger coupler: it allows mechanical linkage between the selected parts and duplicates shock absorption and alignment stability during load conditions.

Cylindrical Rod

The viscosity measurement system in the developed prototype used a cylindrical rod as its rotating element. The rod manufacturer used a ceramic-based substance due to its adequate thermal resistance with minimal density and its silicone oil test fluids' non-reactivity.

FIGURE 3.5 CYLINDRICAL ROD

The rod measures 0.3 m in length and 0.024 m in diameter evenly across its span to support enough testing depth adjustments. Makeup and preparation of the rod surface focused on maintaining smoothness to reduce disturbances from fluid flow and ensure laminar flow throughout its length.

This rod provides acceptable results for ambient-testing fluids, but its properties constrain its use at high temperature conditions that would include molten glass measurements. System updates in the future will include switching the rod to high-temperature-resistant materials to improve the viscometer's operational capabilities.

Together, these components—the Fusion 360-designed and Amazon-sourced motor bracket, the flexible shaft coupler between the motor and sensor, and the large-bore coupler between the sensor and rod—form a well-aligned, mechanically stable assembly. The system configuration enables controlled and precise torque transmission from the motor, which results in essential operational functionality for the viscometer despite minimal losses.

3.3.2 Stepper Motor

A stepper motor is a type of DC electric motor that divides a full rotation into a large number of equal steps. The motor's position can be controlled precisely without any feedback mechanism (open loop). Stepper motors can be viewed as electric motors without commentators. Typically, all windings in the

FIGURE 3.6 STEPPER MOTOR

motor are part of the stator, and the rotor is a permanent magnet or, in the case of variable reluctance motors, a toothed block of some magnetically soft material. All of the commutation must be handled externally by the motor controller, and typically, the motors and controllers are designed so that the motor may be held in any fixed position as well as being rotated one way or the other. Stepper motors function through unique operational methods since they activate timed voltage pulses that drive the motor windings. Each pulse rotates the rotor by an exact degree. The sequence of electrical pulses is called "steps," leading to the name "stepper motor".

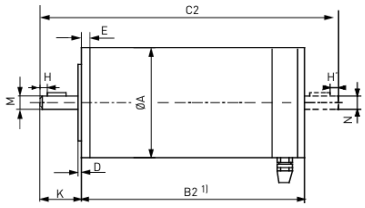


FIGURE 3.7 SCHEMATIC OF STEPPER MOTOR

Degrees per step specifies the number of degrees the shaft will rotate for each full step. A half-step operation of the motor will double the number of steps/revolution and cut the degrees-per-step in half. The degrees per step can be calculated by dividing 360 by the number of steps in 1 complete revolution. Common degree/step numbers include 0.72, 1.8, 3.6, 7.5, 15, and even 90. Degrees per step are often referred to as the resolution of the motor [35].

For this project, I am utilizing the Phytron ZSH 57/2.200.2,8 stepper motor is a robust two-phase hybrid stepper motor with a 1.8° step angle and precise 200-step control, delivering up to 1.25 Nm of holding torque. Step angle of 1.8° (200 steps per revolution), which ensures fine positioning accuracy (±3% relative to the step angle). This product includes mechanical protection up to an IP54 rating and, optionally, IP68 protection and operates successfully in environments with temperatures ranging from +80°C up to +100°C, thus making it suitable for demanding industrial applications. The product offers various wiring setups alongside accessory solutions that increase its capability to handle harsh environmental requirements[36]. This motor delivers reliable, smooth, precise positioning because engineers implemented precision and robustness during its development for tough operational settings. The device proves most effective in applications that require high-performance motion management, together with temperature conditions above room temperature. The motor reaches smooth rotational accuracy through microstepping and performs precise measurements by reducing vibrations. The microstepping capability depends on a motor driver, which follows an explanation in the subsequent section.

TABLE 1: DIMENSIONS OF FIGURE 7 (MM)

A	B2	C2	D	E	F	M	N	P	Q
56.5	102	134	1.5	5	5.3	6.35	6.35	38.1	47

[35] (Tanu Shree Roy)
[36] (GmbH, ZSH Stepper Motor Datasheet)

3.3.3 Stepper Motor Controller MCC-1-32-48-USB-H

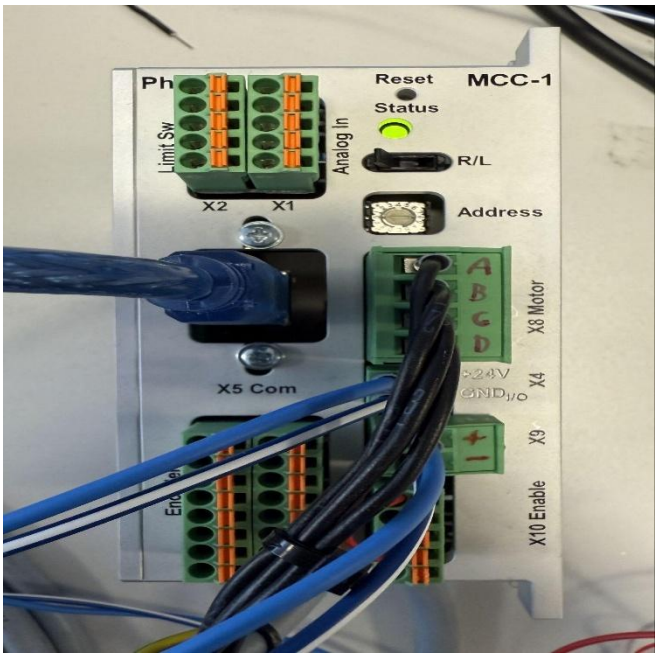


FIGURE 3.5: STEPPER MOTOR CONTROLLER MCC-1-32-48-USB-H

The MCC-1, phytron's freely programmable dual axis stepper motor controller, is a compact stand-alone unit for 2 phase stepper motor providing up to 3.5 Apeak phase current. Controllers in the MCC series have many inputs and outputs (digital and analog). Due to the variety of available host interfaces (USB, Ethernet, etc.), the MCC can be quickly and easily integrated into existing applications.

The Figure shows the Specifications of the Stepper Motor Controller:

Mechanical	
Dimensions (W x H x D)	55 x 127 x 110 mm; 73 x 127 x 110 mm with attached USB converter or terminal adaptor
Weight	Approx. 660 g
Mounting	Wall or rail mounting
Features	
Stepper motors	Suitable for the control of 2 phase stepper motors with 4, (6) or 8 lead wiring
Supply voltage	Controller and motor: 24 to 48 V_{DC}; Limit switches and outputs: 24 V_{DC}
Phase current	Up to 3.5 A_{PEAK}
Step resolution	1/1, 1/2, 1/4, 1/5, 1/8, 1/10, 1/20; for smoother motor rotation: 1/32, 1/64, 1/128 up to 1/256 step of a full step
Step frequency	40,000 steps/sec
Hardware error detection	• Short circuit (between phase and power supply; between both phases; within a motor against ground) • Over temperature • Under voltage
Cable length	Motor: shielded: 50 m max. Signal: shielded: 100 m max.

FIGURE 3.8 SPECIFICATIONS OF THE CONTROLLER

For this project, the Phytron: MCC-1-32-48-USB-H is the stepper motor controller. It is a one-axis stepper motor control unit with integrated power stages and Bipolar control of 2-phase stepper motors. This device features an IP 20 protection rating and runs between +5 to +50 degrees C, which makes it suitable for glass sector applications. The host interface is USB, connecting directly to the Raspberry Pi, with its communication protocol and details explained in the following section[37].

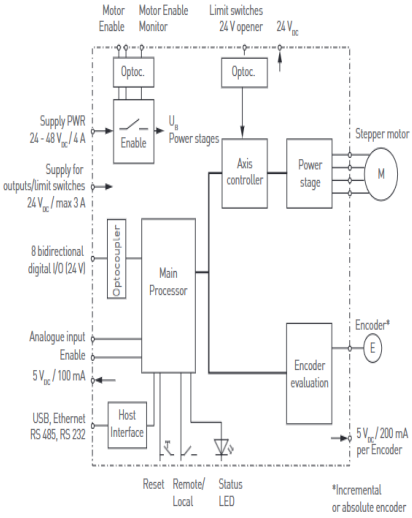


FIGURE 3.9 BLOCK DIAGRAM OF THE CONTROLLER

3.3.4 Dynamic Rotary Torque Sensor

In the construction of the rotational viscometer, accurate torque measurement is critical to determining the viscosity of test fluids. The torque sensor detects the resistance exerted by the fluid on the rotating spindle and converts this into a measurable signal. For this purpose, a ZHKY2050 dynamic rotary torque sensor with a maximum capacity of 0.5 Nm was selected. The sensor operates through a touchless push-pull output method that delivers torque-proportional frequency signals.

FIGURE 3.10 TORQUE SENSOR ZHKY2050

The device consists of an aluminum body coupled with a stainless steel shaft design to ensure reliability while allowing precise measurements (up to ±0.1% F.S) that default to a 10 ± 5 kHz frequency

[37] (GmbH, Stepper Motor Controller MCC-1-32-48-USB-H Datasheet)

output. Its built-for-purpose mechanical structure works at 8000 RPM standard speed using customizable features for rotational viscometer Torque measurement.

3.3.5 Multimeter

The digital multimeter(Figure 3.9) served to conduct voltage tests throughout both the setup and testing periods. The Voltcraft VC155 device serves as a measurement instrument because it can check voltage levels and also measure current output and resistance values and identify discontinuities in electrical systems.

FIGURE 3.11 MULTIMETER

The multimeter primarily served to check the DC voltage at different parts throughout the electrical circuit. Voltage divider output measurements served as the main purpose, where technicians scrutinized the output of a 5V source being stepped down to 3.3V through the divider circuit. The device needed voltage conversion for operating with equipment that receives signals from a lower voltage range. A measurement of the lower resistor's output in the divider showed stable results close to 3.3V, which verified the functioning of the circuit properly.

A check of the 24V DC voltage supply delivered by the Switched-Mode Power Supply (SMPS) to both the motor controller and torque sensor was performed to verify proper voltage transmission. Wiring protocols and electrical polarity received checks throughout different system sections, where unexpected voltage changes required troubleshooting based on measurement results. The multimeter used a rotary selector, which was adjusted to display DC voltage (V⎓) according to the specific section of the circuit being evaluated (logic-level with an optimal range of 20V while the main power supply was checked at 200V. The test probes are connected to the COM and VΩmA ports according to the figure while ensuring stable contact during metric acquisition.

The development of the system relied on this multimeter to check voltage levels in addition to detecting mistakes in wiring and validating electrical connections, as well as the voltage divider function. The multimeter provided secure operation of the viscometer system through this usage method.

.

3.3.6 Thermometer

The process of building and testing the rotational viscometer prototype needed an exact method for temperature measurement because silicone oils exhibit significant variation in their viscosity based on temperature changes. The said experiments required a digital thermometer to measure sample temperature in silicone oils of different viscosities during test cycle operations.

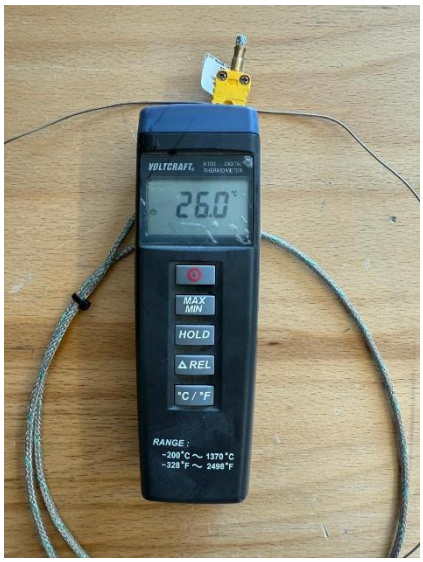


FIGURE 3.12 THERMOMETER

The selected thermometer utilized stainless steel probes (e.g., “as shown in Figure 3.10”), which enabled direct immersion of the sample container through which it collected temperature data in real time. The experiments required the thermometer probe to stay at a fixed depth within the geometric center of the fluid volume for obtaining reliable measurement data. Viscosity measurements required the digital display of the thermometer, which provided resolution at ±0.1°C intervals to verify temperature stabilization achievements before every reading.

Experimental temperature control became mandatory because viscosity has a proven exponential relationship with temperature within studies of complex fluids and lubricants. Zhang (2021) explains that temperature increase causes viscosity to decrease substantially because intermolecular forces, including dipole-dipole interactions and London dispersion forces, become less powerful. The temperature-viscosity relationship for both associated and non-associated liquids can be accurately modeled through logarithmic or exponential mathematical representations, according to Cornelissen and Waterman (1955).

3.3.7 Switching Mode Power Supply

This project utilized an SMPS (Switched-Mode Power Supply) component as an adjustable DC power source to supply energy to essential rotational viscometer system elements. The stepper motor, along with the motor controller, joined by the torque sensor, represented the main system components. The laboratory-grade Manson HCS-3302 DC power supply was chosen because it enables users to set output between 1 to 32V DC, and it supports up to 15A of current.

FIGURE 3.13 SWITCHING-MODE POWER SUPPLY

The rotational viscometer functions reliably at 24V steady output to operate the stepper motor and its controller as well as the torque sensor. A no-load condition on the power supply leads to terminal voltage readings of 23.9V and minimal current draw of 2.5A, as shown in the provided figure, according to the front panel. The operating parameters behave dynamically because of the applied mechanical motor load and torque sensor reactions.

The voltage and current adjustments were possible independently with the SMPS, which had rotary controls that included both coarse and fine-tuning features. The protective systems within the device controlled current movement while searching for short circuits and checking internal heat levels. The safety features protect both the normal function and operational integrity of torque sensors among sensitive devices. Power transmission occurred through banana plug terminals found on the front panel while connecting wires from motor drivers along with sensors.

A dependable, stable power supply became essential because the rotational viscometer system demanded it due to its sensitivity against voltage fluctuations. The precision of torque measurements, together with the controlled stepper motor function, depended heavily on the power supply's reliability. The dependable operation of the SMPS delivered direct support to the data acquisition process, which measured viscosity.

3.3.8 Raspberry Pi 5

The Raspberry Pi single-board computer underwent extensive development while simultaneously addressing affordability in personal computing to support embedded system project development. Researchers at the University of Cambridge initiated the Raspberry Pi device in 2006 before its 2012 launch through the Raspberry Pi Foundation. The foundation created this device to provide students with an economical platform for computing education. The development of this platform led to sustained improvements of both hardware and software components, which now support educational needs and complex industrial applications across different domains.

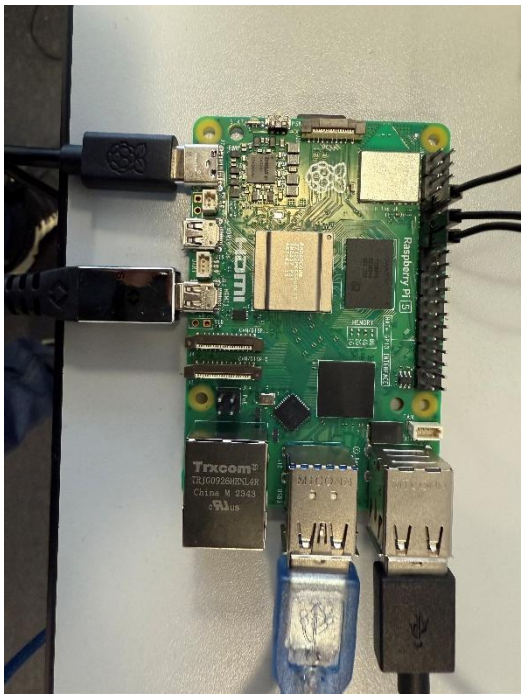


FIGURE 3.14 RASPBERRY PI 5

3.3.9 Raspberry Pi 5 Operating System and Modes of Operation

The rotational viscometer selects Raspberry Pi 5 as its core control system because this device was designed explicitly to work in embedded and industrial operations. The Raspberry Pi 5 functions as the system control because it provides a combination of powerful computing features and complete GPIO management, along with support for open-source software.

The rotational viscometer operates using Raspberry Pi OS (64-bit) because this distribution functions under the Debian base and maintains software support from the Raspberry Pi Foundation. The minimal operating environment of the system delivers stable performance with customizable data collection capabilities for motor control and graphical interface visualization functions. Raspberry Pi 5 represents an exceptional option for sensor integration with microcontrollers since its platform demonstrates high efficiency for processing data between sensors and advanced computations. The

excellent combination of real-time data processing capabilities with top producer/consumer speed recommends this solution to applications needing sensor-based solutions.

The research benefits from the advanced features of Raspberry Pi 5 because they allow sophisticated signal processing in sensors, which leads to better embedded system detection capabilities.

Raspberry Pi's evolution from its initial stages to the superior Raspberry Pi 5 model fulfils the research purpose of enhancing sensor systems and microcontroller connections for contemporary embedded applications.

3.3.10 Control Raspberry Pi 5

The Raspberry Pi 5 can be accessed in different ways.

1. Direct Access (Standalone Mode)

Raspberry Pi 5 functions in standalone mode when users connect it directly to a combination of a monitor through HDMI, as well as a keyboard and mouse.

- Monitor (via HDMI)
- Keyboard
- Mouse

We can view and operate the system through this setup because of its design. Upon Raspberry Pi OS boot completion, users will reach the desktop environment, which enables immediate launching of the viscometer GUI application. A laboratory environment that needs autonomous operation and network independence should use this configuration. The user controls the entire interface of the viscometer while changing experimental parameters and monitoring motor control and torque, and viscosity data in real time using only their device without additional hardware.

2. Remote Access via SSH

The system allows users to control the device through SSH (Secure Shell), which provides access to a headless operation in scenarios where direct access is impractical. The Raspberry Pi operates under remote control because it works with a host PC on a local network. A Secure Shell interface enables users to operate terminals for theoretical procedures.

- Run experiments
- Access logs
- Perform maintenance tasks

The system enables remote management of graphical user interface applications through either X11 forwarding technology or VNC.

Users access Raspberry Pi remotely via the standard SSH remote login command:

“ssh pi@<Raspberry_Pi_IP>”

The SSH client connection command, which is also referred to as an SSH login command, serves this function.

Users can use SSH as a command-line interface which opens Secure Shell (SSH) sessions.

- pi: The username of the Raspberry Pi (default for Raspberry Pi OS).
- <Raspberry_Pi_IP>: Placeholder for the IP address of the Raspberry Pi on the network

3. Remote GUI Access via Raspberry Pi Connect

Through the newly released Beta version of Raspberry Pi Connect, the system provides web browser access to the Raspberry Pi desktop and terminal functionality across the world without requiring complicated network setups [38].

Any user can access the Raspberry Pi graphical interface through connect.raspberrypi.com without requiring special network arrangements.

- End-to-end encrypted communication

Users can access the shell environment through remote terminals, in addition to having an optional screen-sharing option.

- No need for a static IP or VPN setup
- Compatible with Raspberry Pi OS Bookworm or newer versions

The bash command:

(sudo apt update

sudo apt install rpi-connect

rpi-connect on)

3.3.11 Python Programming Environment

The rotational viscometer control system received support through the implementation of Python programming because of its flexible design, simple device connection capabilities, and abundant scientific resource libraries. The created implementation system contained functionality for serial connection control, GPIO sensor connection, real-time data collection, torque and viscosity measurement capabilities, graphical user interface design, and data presentation functions. The viscometer software received support from PyQt5 and matplotlib libraries, together with gpiozero and pandas during its development phase.

The program architecture followed structured guidelines to achieve modular design alongside maintainability features. The real-time collection of sensor data proceeded through threaded processes, while evaluation by mathematical models led to the visualization of data through an integrated plotting panel. The software generates results in an Excel file and JSON format, which could

[38] (Foundation)

be used for evaluation and calibration functions. The exact and efficient operation of the laboratory prototype depended heavily on this software-based methodology.

3.3.12 Integrated Development Environments (IDEs)

The project used two primary Integrated Development Environments (IDEs) for development purposes, which supported different development phases.

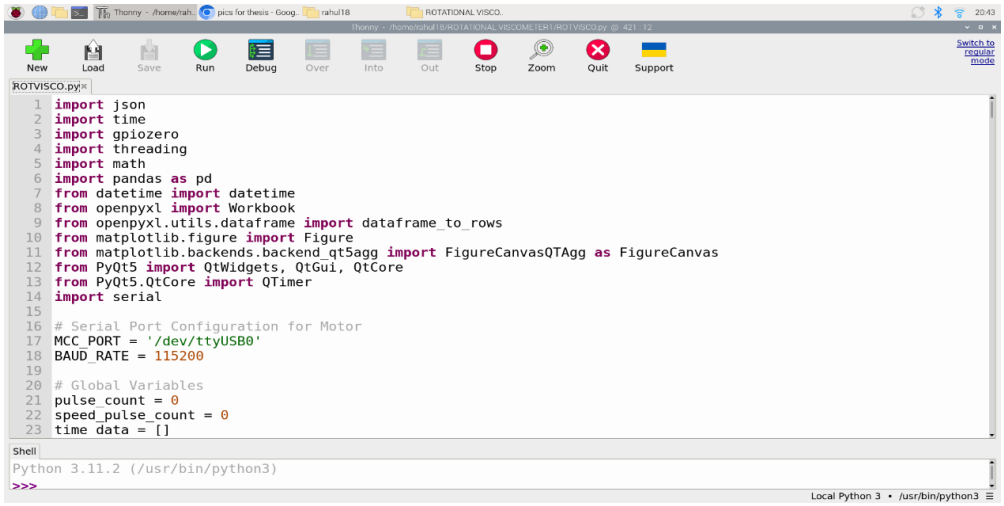


FIGURE 3.15 THE INTERFACE OF THONNY IDE ON RASPBERRY PI IOS

The team adopted PyCharm IDE for debugging purposes, together with calibration coding and error testing needs. The system's calibration was facilitated by using PyCharm's additional features for version control and code navigation, and profiling to achieve complete analysis according to the details in Chapter 6 (Calibration). The software project management became more efficient because this tool helped create detailed systems documentation while providing effective project control.

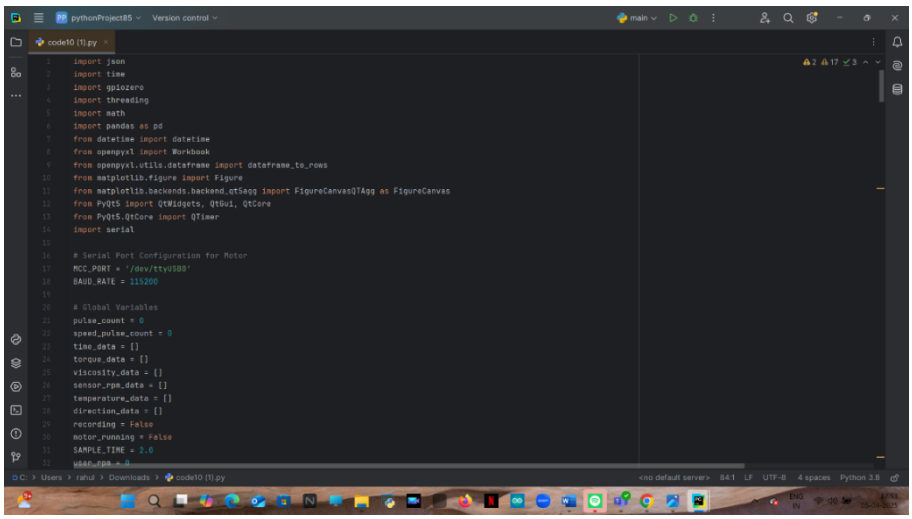


FIGURE 3.16 THE INTERFACE OF THE PYCHARM IDE

Thonny and PyCharm incorporated together provided an optimized software development solution which blended embedded control functionality of the viscometer with analytical requirements for system validation.

3.4 Integration of Stepper Motor to Raspberry Pi through Driver

The Phytron ZSH 57/2.200.2,8 stepper motor is integrated with the Raspberry Pi 5 using the Phytron MCC-1-32-48-USB-H motor controller. As a standalone device, the controller uses its pre-installed proprietary MINILOG serial commands through USB to execute programmed motions and Communication Interface

The MCC-1 controller is connected to the Raspberry Pi via USB and appears as a virtual serial port (e.g., /dev/ttyUSB0).

The Raspberry Pi opens a serial connection to the MCC driver using:

“ser = serial.Serial('/dev/ttyUSB0', baudrate=115200, timeout=2)”

The baud rate is set to 115200 as per the configuration for fast communication.

Communication is established using ASCII-based telegrams formatted as:

“<STX><Address><Instruction><ETX>”

where <STX> and <ETX> are the start (0x02) and end (0x03) of text characters, respectively.

No checksum is required by default, simplifying message construction (GmbH, Programming Manual for MCC Minilog).

3.4.1 Control via MINILOG Commands

The stepper motor receives control through MINILOG instructions that travel from the Raspberry Pi using a Python script and the pyserial library. The motor controller receives execution instructions through direct commands.

Control commands include:

TABLE 2 MINILOG CONTROL COMMANDS OF THE MOTOR CONTROLLER

COMMANDS	DESCRIPTION
XL+	Move the steps in a clockwise direction
XL-	Move the steps in a counterclockwise direction
XS	Stop movement
XMA	Activate power stage (enable motor)
XMD	Deactivate power stage (disable motor)
XP45S1	Full step mode (200 steps)
XP45S2	Half step (400 steps)
XP14S<frequency>	Set the frequency corresponding to the desired RPM

The stepper motor is interfaced with the Raspberry Pi using the Phytron MCC-1-32-48-USB-H motor controller through a USB-based serial connection. Communication is handled in Python using the serial.Serial() object, with the motor controller receiving ASCII-formatted MINILOG commands. Upon user input through the GUI, the desired speed and direction are captured and passed to the control_motor(rpm, direction) function.

This function internally sets the motor to full-step mode using send_command("0", "XP45S1"), calculates the corresponding step frequency using rpm_to_frequency(rpm), and sends directional commands like XL+ or XL- depending on the selected direction The operation sends commands to helper function send_command(address, command) to envelop instructions with control characters including \x02 STX and \x03 ETX. The XS command triggers from the stop_motor() function to stop the motor.

Through its modular design approach, the controller allows motor parameter adjustments through GUI interfaces, which do not affect the controller configuration[39].

Speed and Position Control

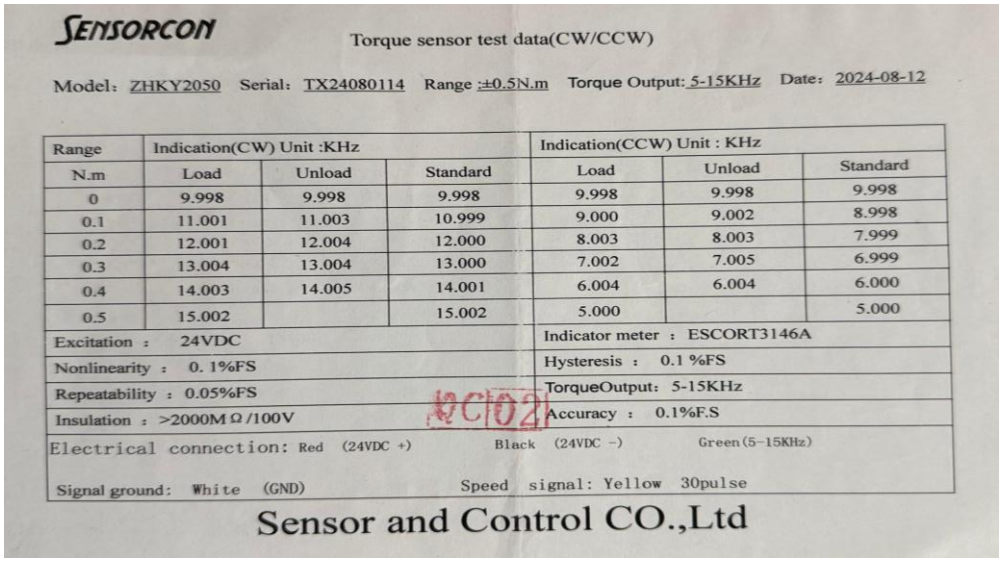
The stepper motor operated in bipolar mode according to the wiring instructions from its datasheet specification. The motor achieved detailed angular control because it operated with a standard step angle measurement of 1.8° for each step. The speed adjustment worked through changing the stepping frequency sent by the controller instead of modifying the supply voltage. The motor's speed was controlled through a constant voltage supply by changing the number of command pulses per second. The rotational speed of the motor increases with rising pulse frequency yet slows down when the pulses decrease.

Achieving accurate angular motions occurred through the transmission of a predefined number of motor steps. A revolution needed 200 pulses because each step led to a 1.8° rotation. The system achieved accurate positioning by determining and monitoring the number of sent pulses. The system used this method to deliver a straightforward method for motor speed and position control, which maintained the reliable operation of the prototype system.

[39] (GmbH, Programming Manual for MCC Minilog)

3.5 Integration of Torque Sensor to Raspberry Pi 5

The ZHKY2050 Torque sensor designates itself as a small-size, precise rotary torque sensor that operates effectively at high speeds for uninterrupted rotation sequences. This sensor creates a frequency output signal that expands linearity to the torque force without physical contact between its components. The sensor measures up to 0–0.5 Nm of torque. The calibrated data given by the manufacturer is represented in Figure 3.17:



SENSORCON

Torque sensor test data(CW/CCW)

Model: ZHKY2050 Serial: TX24080114 Range :±0.5N.m Torque Output: 5-15KHz Date: 2024-08-12

Range	Indication(CW) Unit :KHz			Indication(CCW) Unit : KHz		
N.m	Load	Unload	Standard	Load	Unload	Standard
0	9.998	9.998	9.998	9.998	9.998	9.998
0.1	11.001	11.003	10.999	9.000	9.002	8.998
0.2	12.001	12.004	12.000	8.003	8.003	7.999
0.3	13.004	13.004	13.000	7.002	7.005	6.999
0.4	14.003	14.005	14.001	6.004	6.004	6.000
0.5	15.002		15.002	5.000		5.000

Excitation : 24VDC	Indicator meter : ESCORT3146A
Nonlinearity : 0. 1%FS	Hysteresis : 0.1 %FS
Repeatability : 0.05%FS	TorqueOutput: 5-15KHz
Insulation : >2000MΩ/100V	Accuracy : 0.1%F.S

Electrical connection: Red (24VDC +) Black (24VDC -) Green(5-15KHz)

Signal ground: White (GND) Speed signal: Yellow 30pulse

Sensor and Control CO.,Ltd

FIGURE 3.17 DATA SHEET OF ZHKY2050 **TORQUE SENSOR FOR CALIBRATION**

Frequency output from the sensor increases linearly with the torque. The speed output of the sensor reaches 30 kHz, which generates 60 speed pulses for each revolution. Hence, a frequency of 1 kHz equals 1000 RPM. The sensor provides two outputs, which enable simultaneous measurements of torque and speed data. This device requires a 24 V DC supply voltage and demonstrates durability, along with low noise and reliable digital signals under dynamic operations (Sensor and Control Co.).

3.5.1 Push-Pull Output Characteristics

The ZHKY2050 torque sensor provides a push-pull (active-high) square wave output for both torque and speed channels. In a push-pull configuration:

- One internal transistor **“**pushes” the signal high (toward +5V)
- Another **“**pulls” it low (toward the ground)

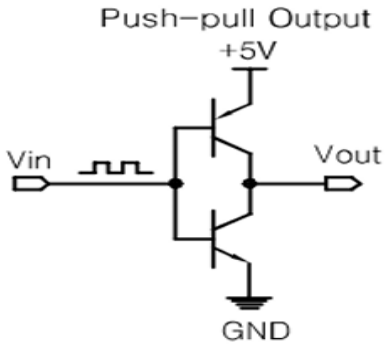


FIGURE 3.18 PUSH-PULL OUTPUT SIGNAL OF ZHKY2050

The torque sensor generates a digital square wave output signal through a push-pull interface, which increases its frequency as the torque or speed rises. The GPIO interrupt system included in the Raspberry Pi detects each upward transition of the waveform. The number of recorded pulses within 2 seconds of sampling produces the frequency calculations (Equation 11).

EQUATION 11

$$f = \frac{pulse\ count}{sample\ time}$$

The linear transformation converts the identified frequency (f) value to torque values:

EQUATION 12

$$T(Nm) = 0.1 * f_{khz} - 1$$

At a frequency of 9.998 kHz, the torque output reading is zero Nm, while it reaches 0.5 Nm at 15.002 kHz.

The speed measurement process happens on GPIO 21 using the frequency (f_{speed}) output to calculate RPM directly.

EQUATION 13

$$RPM = \frac{f_{speed}}{60} * 1000$$

The device generates digital square wave signals that oscillate between 0 V and +5 V, while the output frequency matches the actual measurement values of torque or speed. Through green torque cables and yellow speed cables, and white common wires, the unit transmits signals with the red and black wires used for a 24 V power supply, whose logic is internally regulated to 15 V.

8 pins electrical wiring (output signal coexist with communication)					
Power supply	pin 5	Red	Power supply Vin+		
	pin6	Black	Power supply Vin-		
Signal			Pulse / current / voltage		Pulse Differential
	pin3	Green	Torque		Torque A+
	pin4	Yellow	Speed		Speed B+
	pin1	White	Signal - (GND)		Torque A -
	pin2	Blue	/		Speed B-
Communication			RS485	RS232	CAN
	pin7	Brown	A+	TxD	H+
	pin8	Grey	B-	RxD	L-
Signal - （GND）Have to use white GND, cannot use input power supply -					
Vin- and GND are not same active output, has voltage difference					

FIGURE 3.19 ELECTRICAL WIRING AND OUTPUT SIGNAL CONNECTION OF ZHKY2050

3.5.2 Signal Conditioning Circuit

Since the Raspberry Pi GPIOs operate at 3.3 V logic levels, direct connection of the sensor's 5 V output is not advisable. To safely interface the sensor with the Raspberry Pi, a signal conditioning circuit is used, e.g., Figure 3.20. This circuit comprises two key stages:

a. Voltage Divider

A resistive voltage divider reduces the 5 V logic signal from the sensor down to approximately 3.3 V. For both torque and speed channels, the divider consists of:

- $5k\Omega$ resistor (series)
- $10k\Omega$ resistor (to ground)

The resulting output voltage is:

EQUATION 14

$$V_{out} = 5V \times \frac{10k\Omega}{5k\Omega + 10k\Omega} \approx 3.3V$$

b. RC Band-Pass Filter

To further clean the signal and remove noise or unintended harmonics, an RC band-pass filter is applied after the voltage divider. The filter is tuned to pass only the relevant frequency range for each signal:

For Torque signal/Speed signal:

EQUATION 15

$$f_1 = \frac{1}{2 \times \pi \times R_1 \times C_1}$$

EQUATION 16

$$f_2 = \frac{1}{2 \times \pi \times R_2 \times C_2}$$

TABLE 3 DETAILS REGARDING CIRCUIT CONNECTION

Signal	GPIO Pin	Frequency Range	Filter Components
Torque	GPIO 17	4–16 kHz	$5nf$, $5k\Omega$, $8k\Omega$, $2nf$
Speed	GPIO 21	2–32 kHz	$10nf$, $1k\Omega$, $8k\Omega$, $5nf$

This figure illustrates the signal conditioning circuit for integrating torque sensor output to raspberry pi:

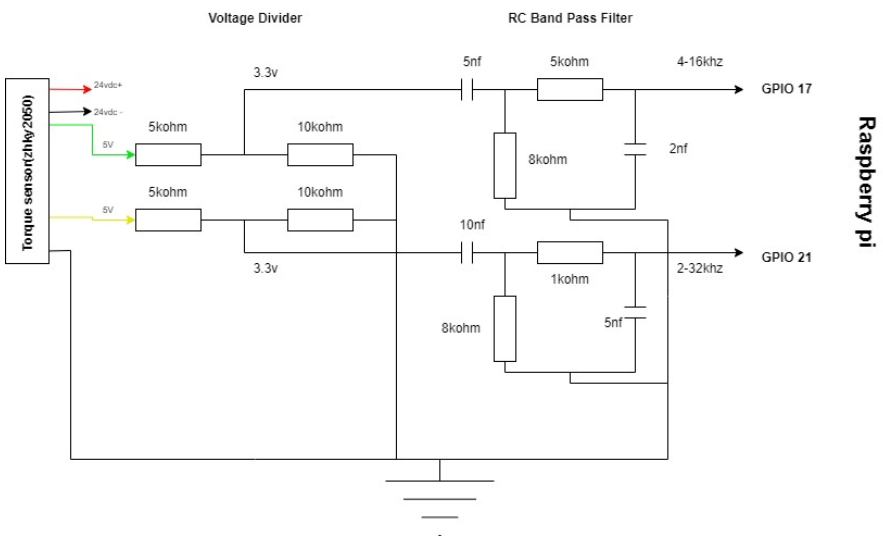


FIGURE 3.20 THE SIGNAL CONDITIONING CIRCUIT WITH VOLTAGE DIVIDE AND RC BAND FILTER

3.6 Graphical User Interface (GUI) for Real-Time Control and Monitoring

The rotational viscometer operates through a user-friendly Graphical User Interface (GUI) built using the PyQt5 framework. Through a Graphical User Interface, the stepper motor can be experimental settings, and view immediate torque measurements in addition to monitoring instantaneous viscosity data and producing result outputs.

The GUI enables its operation within the Raspberry Pi environment by showing sensor data from ZHKY2050 torque sensors and control of stepper motors in real time.

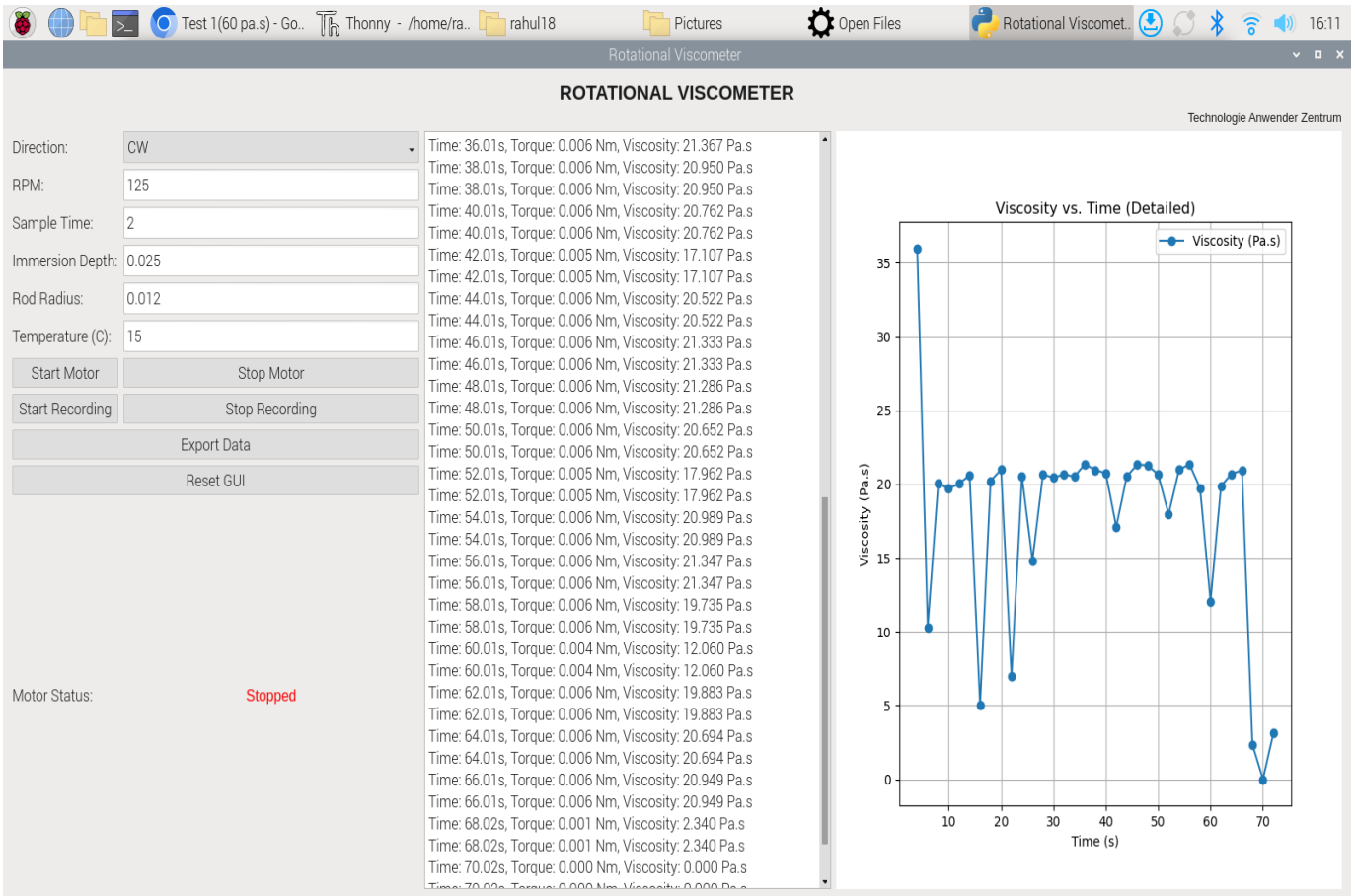


FIGURE 3.21 GRAPHICAL USER INTERFACE

3.6.1 GUI Structure and Components

The GUI has 3 parts:

1. User control panel

TABLE 4 DETAILS OF EACH SECTION IN THE CONTROL PANEL OF THE GUI

Direction Selector	Select rotation direction (CW/CCW)
RPM Input	Set motor rotation speed in RPM (user input)
Sample Time	Set the data sampling interval
Immersion Depth	Set the length of the spindle immersed in the liquid (user input)
Rod Radius	Radius of the rod (m)
Temperature	Temperature of the liquid (degrees Celsius)
Start Motor / Stop Motor	Button for start and stop the stepper motor
Start Recording / Stop Recording	Start and stop data acquisition and visualization
Export Data	Exports recorded data to an Excel file along with generated plots
Reset GUI	Clears plots, data, and resets all settings to default

2. Data Display Panel

The data panel shows the live-recorded results in text format:

- Time**,** Torque (Nm)**,** and Viscosity (Pa.s) are printed sequentially.
- The data is continuously updated as new measurements are acquired from the sensor.

3. Real-time plotting panel

Live data visualisation of Viscosity against Time occurs in the right plotting area:

An update of the plot occurs every second through QTimer-based functionality.

The Viscosity vs. Time real-time plot function is managed by Matplotlib embedded within the PyQt interface through Figure Canvas. The observation of dynamic behaviour patterns or smooth states and periodic changes occurs during experimentation through the visual interface.

3.7 Workflow of GUI Process

The real-time data acquisition system within the developed GUI provides continuous operation for experiments. Through GPIO interrupt callbacks, the Raspberry Pi reads sensor pulse signals that enable accurate data counting at high signal frequencies.

The program computes frequency and torque data and viscosity values when each 2-second sampling period ends (default duration). The GUI displays the appended real-time values on its text output for user observation of changing data.

The software displays the viscosity-time data graph during rotation in real time for visual monitoring of fluid behaviour. The real-time display enables several beneficial features, which include monitoring short-lived phenomena while assessing instrumental stability alongside immediate detection of electronic noise and physical disturbances, and sensor breaks. The system displays real-time updated data plots of viscosity changes on a graph that responds instantly to experimental parameter adjustments regarding RPM and rod dimensions, together with immersion depth adjustments.

3.8 Data Acquisition

The graphical user interface for the rotational viscometer received a data acquisition system to automatically manage experimental measurement collection and storage in real time. The hardware and software system is built using the Raspberry Pi 5 with Python programs, which work with the pandas, datetime, and openpyxl libraries for handling data processes and storage, and data export purposes.

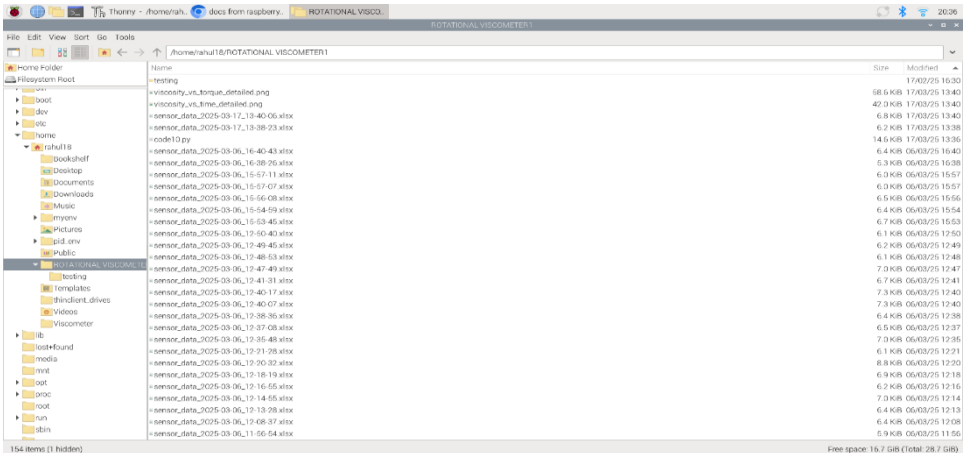


FIGURE 3.22 THE FOLDER VIEW SHOWS STORED EXCEL DATA FILES AND CORRESPONDING PLOT IMAGES.

The test cycle collected continuous sensor output data through GPIO interrupt callback operations, which acquired torque along with time parameters and rotational speed results. The continuous signals entered the system for processing through a sampling time of 2 seconds. The system stored calculated results every 2 seconds that included torque (Nm), viscosity (Pa·s), user-selected RPM, temperature (°C), immersion level, diameter of vessel, and direction inside the memory before adding them to an active dataset. The system generated timestamp-based file names for structured Microsoft Excel files (.xlsx) for automatic measurement storage, e.g., Table 5. The display shows the storage directory in Figure 26, and each file represents a distinct test iteration (see Figure 26). The file format established one observation per row, which enabled future comparison analysis as well as ensured experiment reproducibility. Plots appeared automatically using the matplotlib library while tabulated data existed. After each session completion, the main evaluation plot of viscosity versus time rendered an image file named Figure 3.23.

TABLE 5 EXAMPLE OF DATA COLLECTED IN A SINGLE TEST

Time_s	Torque_Nm	Viscosity_Pa_s	User_RPM	Temperature_C	Direction	Immersion depth	vessel diameter
4.0018496	0.0164	22.04425904	70	21.9	CW	0.08	4.25
6.0076971	0.0215133	27.10201724	70	21.9	CW	0.08	4.25
8.0080697	0.0247090	29.96539698	70	21.9	CW	0.08	4.25
10.008435	0.024512	29.79470957	70	21.9	CW	0.08	4.25
12.008790	0.024668	29.92986033	70	21.9	CW	0.08	4.25
14.009364	0.023756	29.13119368	70	21.9	CW	0.08	4.25
16.009736	0.023409	28.82427187	70	21.9	CW	0.08	4.25
18.010327	0.024997	30.21468285	70	21.9	CW	0.08	4.25
20.011045	0.024032	29.37441247	70	21.9	CW	0.08	4.25
22.011731	0.024498	29.78250456	70	21.9	CW	0.08	4.25
24.012404	0.025055	30.26487106	70	21.9	CW	0.08	4.25
26.013023	0.024432	29.72529148	70	21.9	CW	0.08	4.25
28.013387	0.024913	30.14243749	70	21.9	CW	0.08	4.25
30.014087	0.024391	29.68876887	70	21.9	CW	0.08	4.25
32.014456	0.025110	30.31272881	70	21.9	CW	0.08	4.25
34.015090	0.024175	29.50013493	70	21.9	CW	0.08	4.25

This graph helped to reveal how the fluid behaved when subjected to shear force. Figure 27 shows the scenario of real-time plotting from a test session where the viscosity evolution appeared dynamically in Figure 3.23.

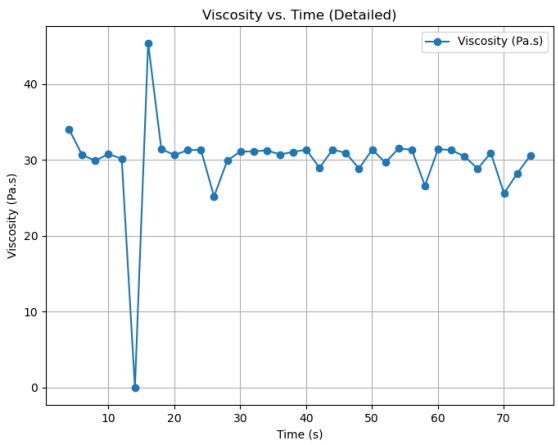


FIGURE 3.23 REAL-TIME VISCOSITY VS TIME PLOT FROM GUI

The automated export method, together with the logging system, decreased the requirement for human operator involvement while creating standardised documentation for every experiment. The graphical user interface managed all data processing operations while abstaining from using external processing applications.

3.9 Workflow of the system

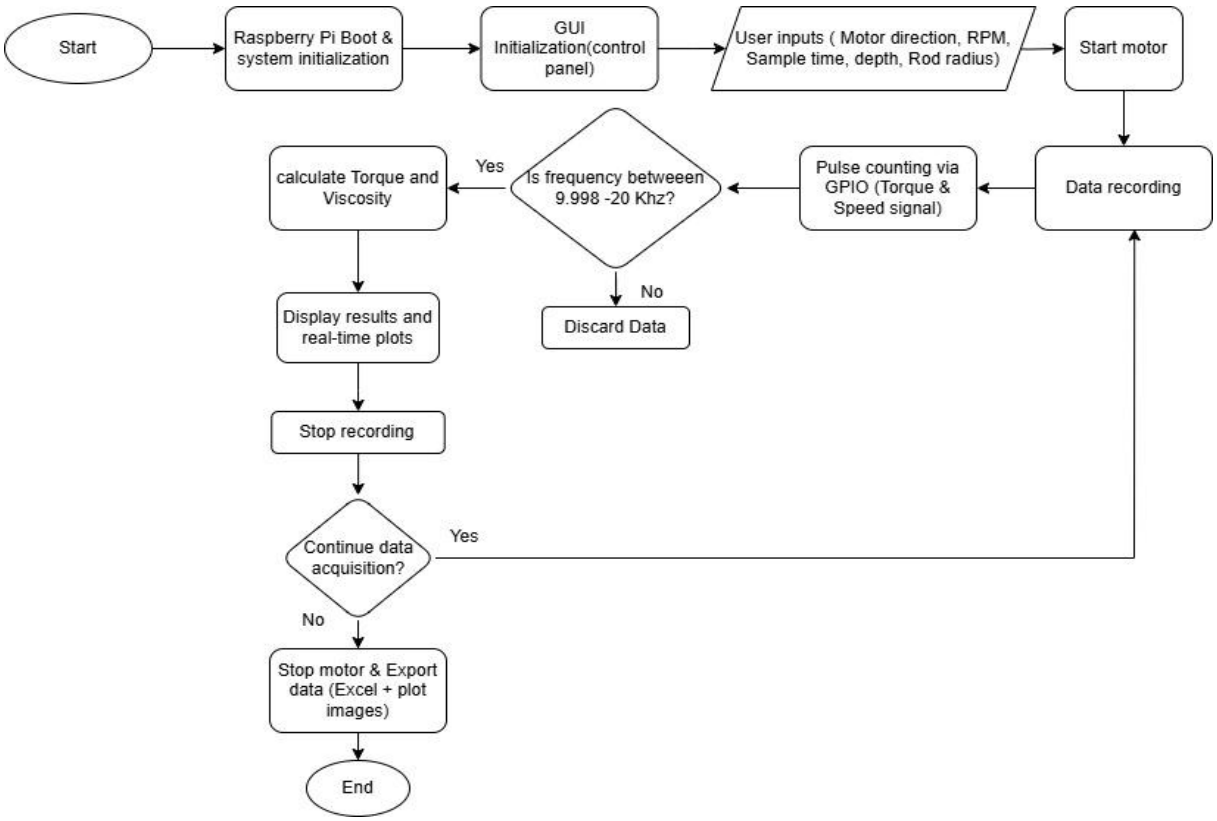


FIGURE 3.24 FLOW CHART OF THE ENTIRE SYSTEM

Chapter 4: Testing and Calibration

4.1 Experimental setup

The experimental setup consists of a rotating rod with a diameter of 0.024 m and a length of 0.3 m. One end of the rod is connected to the shaft of a torque sensor, while the other end of the sensor is connected to the motor shaft. These connections are established using suitable couplings to ensure proper alignment and minimal mechanical losses.

- The torque sensor is securely fixed to a frame using an aluminium plate, ensuring stable measurements.
- The motor is mounted in a holder, which is also firmly attached to the same frame as the sensor, maintaining a rigid structure.

This setup ensures that torque is accurately transmitted from the motor through the sensor to the rotating rod, allowing for precise viscosity measurements.

FIGURE 4.1 MECHANICAL SETUP WITH A SAMPLE FOR A TRIAL RUN

4.2 Trial run without sample

After the successful design, development, and integration of the rotational viscometer system, a test run was conducted to validate the mechanical assembly, electronic interfacing, and software functionality under realistic experimental conditions. This phase is critical to ensure that the system operates as intended before performing actual viscosity measurements.

The first step of the test run involved checking the ability of the system to rotate the motor according to user-defined parameters (RPM and rotation direction) input through the GUI.

4.2.1 Result of the first trial

The system successfully sent the appropriate serial command to the motor driver. The rotation direction was verified by switching between CW and CCW in the GUI, and observing that the actual motion of the shaft and the measured rotational speed matched the user-set RPM within acceptable tolerance.

During the initial test run performed in full-step mode (the default stepping mode of the stepper motor driver), a minor vibration was observed in the spindle during rotation. This vibration, although small, could negatively affect the stability of the measurement.

To address the observed vibrations during the initial test run, the stepping mode of the stepper motor driver was adjusted from full step to half-step mode, thereby implementing microstepping. This measure effectively reduced the mechanical oscillations of the spindle during rotation. The improved smoothness of the motion, resulting from the finer angular resolution of the stepper motor, contributed significantly to the mechanical stability of the system.

4.3 Testing with a sample

When the test run is completed successfully, start with the use of a sample fluid that has similar viscous properties to glass melt. As the setup is at the Laboratory level, for testing purposes, it´s better to use oil like Silicone Oil. The user can place the fluid on the holder under the direction of the ceramic rod, as shown in the figure. With GUI, the user can determine the torque and viscosity at a sampling interval of 2 seconds and visualise a real-time plot of viscosity vs time.

4.3.1 Sample Details

The developed prototype underwent experimental validation through the use of commercially available silicone oils that had different viscosity values.

TABLE 6 DETAILS OF SAMPLES USED FOR TESTING AND CALIBRATION

No.	Silicone Oil	Viscosity (Pa.s)
1	Silikonöl B 10.000 cSt	9.65
2	Silikonöl B 20.000 cSt	19.3
3	Silikonöl B 30.000 cSt	29.1
4	Silikonöl B 40.000 cSt	38.8
5	Silikonöl B 60.000 cSt	58.2

QUAX GmbH from Obernburg, Germany, provided these silicone oils, where the manufacturer provided viscosities which used to compare with their prototype test results.

In this phase of the experiment, two different silicone oils, each with distinct nominal viscosities (40 cSt and 60 cSt), were used as test samples to evaluate the performance of the rotational viscometer.

The choice of silicone oil was motivated by its stable and well-known rheological properties, making it a suitable substitute for glass melt in laboratory conditions.

The tests were carried out under controlled laboratory conditions, varying the rotational speed between 50 and 70 RPM while maintaining a constant immersion depth of 0.05 m. The first set of tests was performed with silicone oil having a nominal (reference) viscosity of 38.8 Pa·s, while the second set involved a higher-viscosity silicone oil with a nominal value of 58.2 Pa·s. The measured torque and the calculated viscosity for each case are presented in Table 7.

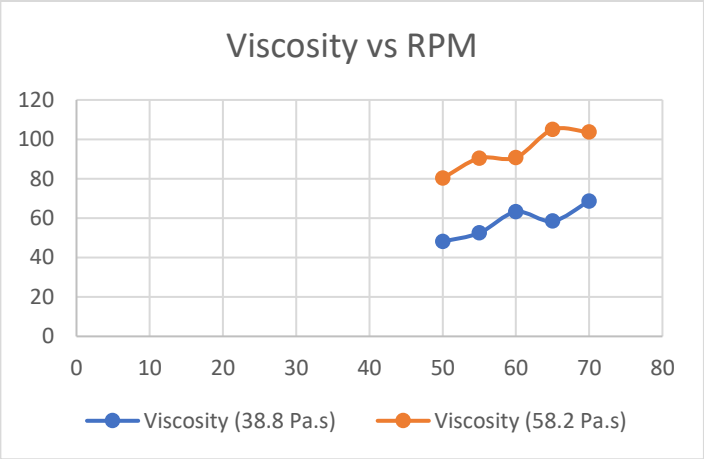


FIGURE 4.2 PLOT OF THE RESULT IN TABLE 7

RPM	Radius of rotor	Immersion depth	Tempeature	Torque	Viscosity	Required Viscosity(Pa.s)
50	0.012	0.05	23.3	0.01139	48.102	38.8
55	0.012	0.05	23.3	0.01367	52.482	38.8
60	0.012	0.05	23.3	0.01796	63.21	38.8
65	0.012	0.05	23.3	0.01802	58.49	38.8
70	0.012	0.05	23.3	0.02277	68.67	38.8
50	0.012	0.05	23.4	0.01900	80.3	58.2
55	0.012	0.05	23.4	0.02353	90.43	58.2
60	0.012	0.05	23.4	0.02833	90.76	58.2
65	0.012	0.05	23.4	0.03231	104.95	58.2
70	0.012	0.05	23.4	0.03436	103.63	58.2

TABLE 7 TEST RESULTS OF SILICON OIL IN TWO DIFFERENT VISCOSITIES

The results clearly show that the calculated viscosity tends to vary with both the rotational speed and the sample properties. Ideally, for a Newtonian fluid such as silicone oil, the measured viscosity should remain constant regardless of the rotation speed. However, significant deviations were observed, highlighting the influence of RPM on the measured values.

These findings further underline the necessity of a proper calibration of the system to minimize the dependence on experimental parameters. The calibration process and its results will be thoroughly described in the following chapter.

FIGURE 4.3 TESTING SAMPLE (SILICON OIL 58.2 PA.S)

4.4 Calibration

The first calibration was performed using a standard fluid with a known viscosity of 58.2 Pa·s. Torque measurements were recorded at seven discrete rotational speeds ranging from 30 RPM to 60 RPM. The experiment was conducted by running the viscometer under controlled conditions, and data were collected from the torque sensor and the motor controller.

4.4.1 Calibration Curve and Equation

To apply this calibration systematically, the calibration factor was plotted against the measured torque using Microsoft Excel (Figure 31). A non-linear regression model was fitted to this data, resulting in a calibration equation:

$$\text{Measured Viscosity} = \boldsymbol{\mu} = \frac{\boldsymbol{T}}{\boldsymbol{\pi} * \boldsymbol{d}^2 * \boldsymbol{h} * \frac{\boldsymbol{\omega}}{\mathbf{2}}}$$

EQUATION 17

$$\text{Calibrated Viscosity1} = \text{Measured viscosity} \times (0.1176 \times T^{0.464})$$

Where T The measured torque during the first test before calibration.

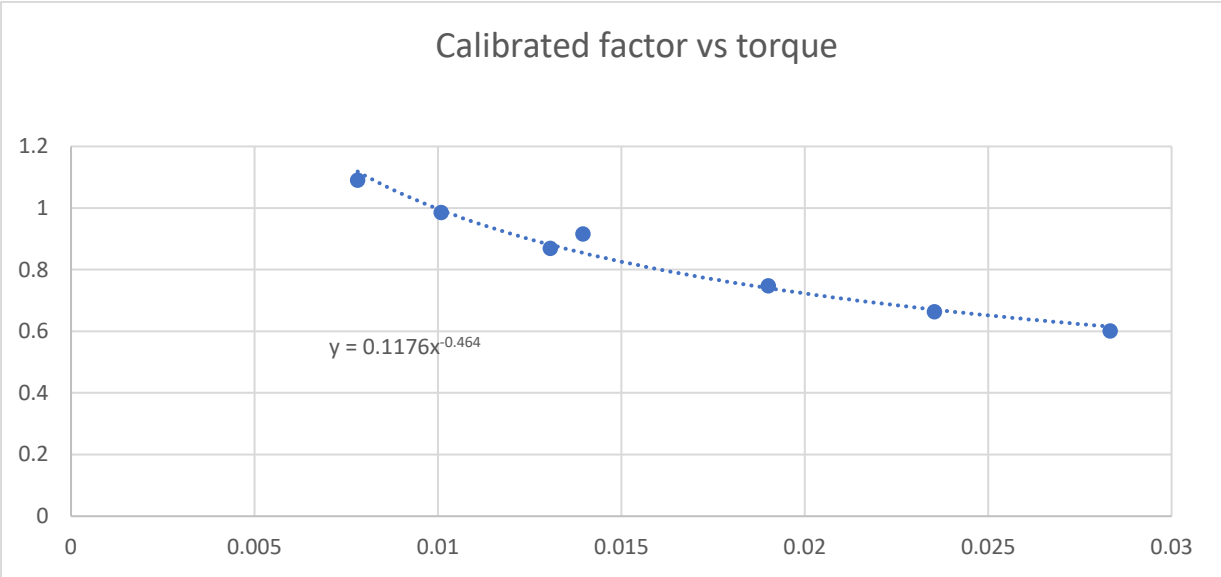


FIGURE 4.4 CALIBRATION CURVE OF THE FIRST CALIBRATION

The calibration factor and calibrated viscosity were computed for each torque setting, as shown in Table 8:

TABLE 8 RESULT AFTER THE FIRST CALIBRATION

RPM	Angular Velocity	Torque Measured	Viscosity measured	Calibrated viscosity	Required viscosity	Calibrated factor
30	3.14	0.007811689	55.02032314	61.47475595	60	1.090506136
35	3.663333333	0.010087749	60.90117856	60.43276642	60	0.985202609
40	4.186666667	0.013066978	69.02630859	60.7458607	60	0.869233793
45	4.71	0.01395245	65.51449987	55.92769623	60	0.915827796
50	5.233333333	0.019002945	80.30643899	59.3997736	60	0.747138097
55	5.756666667	0.02353888	90.43209145	60.56493932	60	0.663481282
60	6.28	0.028330093	99.76913751	61.31411538	60	0.60138838

4.5 Final Calibration- Data Processing and Polynomial Correction

Following the initial calibration procedure, it was observed that although the results showed reasonable accuracy across varying rotational speeds (RPM), notable discrepancies persisted when the immersion depth of the rotating rod was altered. These deviations were particularly evident during testing with some reference fluid (58.2 Pa·s, 38.2 Pa.s, 29.1 Pa.s, 19.3 Pa.s and 9.65 Pa.s). To enhance measurement reliability, a final calibration was undertaken by incorporating different immersion depths into the correction methodology.

4.5.1 Data Collection and Calibration Factor Derivation

A series of measurements was conducted by systematically varying both the immersion depth and RPM. For each condition, torque was measured, and viscosity was calculated using the primary calibration model. Since the test fluid's viscosity was known, a calibration factor was computed for each point using the equation:

EQUATION 18

$$\text{Calibrated Factor} = \frac{Required\ Viscosity}{Measured\ Viscosity}$$

This factor served as a scalar to correct the measured values and compensate for geometric or hydrodynamic inconsistencies introduced by varying immersion depths.

4.5.2 Calibration Curve

The calibration factor values were plotted against their corresponding torque readings, as shown in Figure 4.5. The observed nonlinear trend suggested a fourth-degree relationship. Accordingly, a fourth-order polynomial equation was fitted using numerical regression techniques implemented in Python (numpy.polyfit() and numpy.poly1d() functions). This yielded the calibration model of the form:

EQUATION 19

$$f(x) = -150334 \times T^4 + 27627 \times T^3 - 1712.5 \times T^2 + 50.227 \times T + 0.4264$$

Where: T = Measured Torque

Where 'x' represents the measured torque (T).

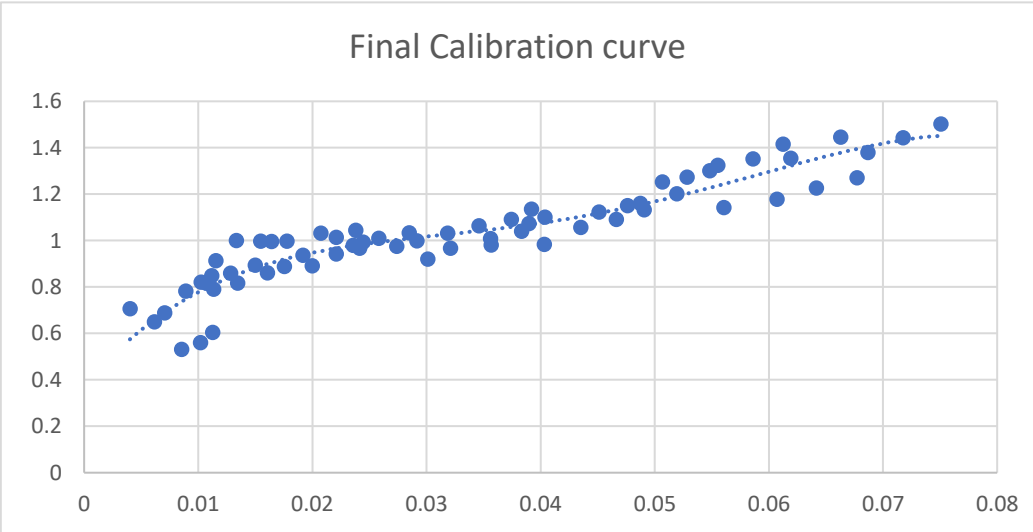


FIGURE 4.5 THIS FIGURE ILLUSTRATES THE PLOT OF THE CALIBRATION FACTOR VERSUS TORQUE

4.5.3 Correction and Viscosity Adjustment

The fitted polynomial equation was then applied to calculate a corrected calibration factor for each torque measurement. This was used to compute the calibrated viscosity by:

Now the final viscosity,

EQUATION 20

$$\text{Calibrated Viscosity2} = \text{Calibrated Viscosity1} \times (f(x))$$

4.6 Python Implementation

The entire procedure was automated using Python. The script includes:

- Importing and handling the Excel dataset (pandas).
- Fitting the polynomial model (numpy).
- Generating calibrated viscosity values.
- (During development) Visualising data and fit curves using matplotlib.

The source code used for the data processing and the original dataset collected for this final calibration have been included in the drive.

This final calibration approach has significantly improved the reliability of viscosity measurements under practical test conditions, especially in setups where immersion depth and RPM vary slightly. The torque-based polynomial correction ensures that the system can compensate for both hydrodynamic and geometric effects in a non-invasive and repeatable manner.

Chapter 5: Results

5.1 Result of Final Testing

The final evaluation of the developed rotational viscometer was conducted through tests with five silicone oils that had previously determined viscosities. Tests were executed with the rotational speed set at 60 RPM, 70 RPM, and 80 RPM while the immersion depths changed between 0.06 m up to 0.08 m to assess the prototype's operational range.

The ZHKY2050 dynamic torque sensor measured torque continuously to analyse viscosity through the calibrated Equation in Chapter 4. The automatic test cycles operated with continuous recording of torque and viscosity measurements at 2-second intervals. The test run final values resulted from averaging all measurements obtained during the experimental procedure for each tested condition. The experimental technique reduced both random noise and temporary variations in the data.

A summary of the averaged results of the final testing of silicon oil with different viscosities in different immersion depths and RPM conditions is provided in Table 9.

TABLE 9 FINAL RESULTS OF TESTING WITH SILICON OIL IN DIFFERENT VISCOSITIES

RPM	Immersion depth	Torque	Measured viscosity (Pa.s)	Required Viscosity (Pa.s)
60	0.06	0.0035716	9.83	9.65
70	0.06	0.0048577	10.75	9.65
80	0.06	0.0043781	8.89	9.65
60	0.075	0.005076	10.38	9.65
70	0.075	0.0058808	10.03	9.65
80	0.075	0.0068345	10.15	9.65
60	0.07	0.009598	18.94	19.3
70	0.07	0.0119542	19.49	19.3
80	0.07	0.0139564	19.36	19.3
60	0.06	0.0087853	20.51	19.3
70	0.06	0.0113781	21.81	19.3
80	0.06	0.0122636	20.31	19.3
60	0.06	0.0137103	29.69	29.1
70	0.06	0.0166868	29.74	29.1
80	0.06	0.0192916	29.07	29.1
60	0.07	0.0156924	28.33	29.1
70	0.07	0.019299	28.48	29.1
80	0.07	0.023265	28.68	29.1
60	0.08	0.0277743	37.87	38.2
70	0.08	0.0351307	38.53	38.2
80	0.08	0.0398774	37.21	38.2
60	0.07	0.0234774	38.43	38.2
70	0.07	0.0308105	40.02	38.2
80	0.07	0.0348473	38.07	38.2
60	0.06	0.0337038	58.13	58.2
70	0.06	0.0406411	57.55	58.2
80	0.06	0.0456192	55.63	58.2
60	0.08	0.0450446	55	58.2
70	0.08	0.0544515	56.97	58.2
80	0.08	0.0612515	56.51	58.2

The comparison between the measured viscosities and the known reference values of the silicone oils demonstrates that the developed system can produce reliable measurements within acceptable tolerances.

For further clarification, the comparison between measured viscosity and required viscosity is plotted in Figure 5.1.

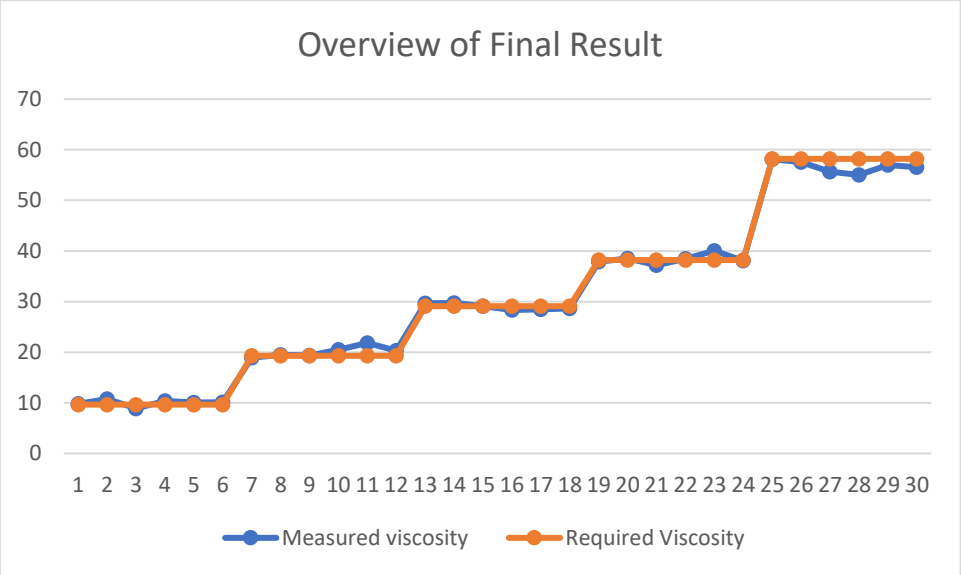


FIGURE 5.1 COMPARISON BETWEEN MEASURED AND REQUIRED VISCOSITY

5.2 Example of final raw sensor data

For better understanding, a table with data of a sample (29.1 Pa.s/30.000 cSt Silicone Oil) in a sample time of 2 seconds shows parameters including RPM, immersion depth, Torque, Viscosity, and Temperature are presented in Table 10. Other parameters included in this test are the radius of the rod (0.012 m) and the vessel radius (4.25 m).

TABLE 10 FINAL RAW SENSOR DATA (29.1 PA.S SILICON OIL)

Time_s	Torque_Nm	Viscosity_Pa_s	User_RPM	Temperature_C	Direction	Immersion depth
6.00769711	0.021513374	27.10201724	70	21.9	CW	0.08
8.00806975	0.024709072	29.96539698	70	21.9	CW	0.08
10.0084355	0.02451265	29.79470957	70	21.9	CW	0.08
12.0087905	0.024668119	29.92986033	70	21.9	CW	0.08
14.0093648	0.023756003	29.13119368	70	21.9	CW	0.08
16.0097365	0.023409802	28.82427187	70	21.9	CW	0.08
18.0103273	0.024997215	30.21468285	70	21.9	CW	0.08
20.0110459	0.024032069	29.37441247	70	21.9	CW	0.08
22.0117319	0.024498633	29.78250456	70	21.9	CW	0.08
24.0124042	0.025055407	30.26487106	70	21.9	CW	0.08
26.0130231	0.024432971	29.72529148	70	21.9	CW	0.08
28.013387	0.024913555	30.14243749	70	21.9	CW	0.08
30.0140877	0.024391098	29.68876887	70	21.9	CW	0.08

5.3 Error Analysis

To analyse the accuracy and stability of the system, an error calculation is performed on the measured data.

For each test of the sample, the standard deviation and mean were calculated. This procedure helps to understand the consistency and provides details about noise and variation.

An overview of the error calculation is presented in Table 11.

TABLE 11 MEAN AND STANDARD DEVIATION FOR TORQUE AND VISCOSITY

Silicone Oil	Torque (Nm) (Mean ± Std)	Viscosity (Pa·s) (Mean ± Std)
9.65 Pa·s	0.00445 ± 0.00142	9.92 ± 2.88
19.3 Pa·s	0.01396 ± 0.00117	19.36 ± 1.35
29.1 Pa·s	0.02379 ± 0.00215	29.11 ± 2.04
38.8 Pa·s	0.03192 ± 0.00209	38.69 ± 1.85
58.2 Pa·s	0.03343 ± 0.00174	57.79 ± 2.21

It was observed that for most test conditions, the standard deviations were relatively small compared to the mean values, indicating good measurement repeatability. E.g., Figure 5.2.

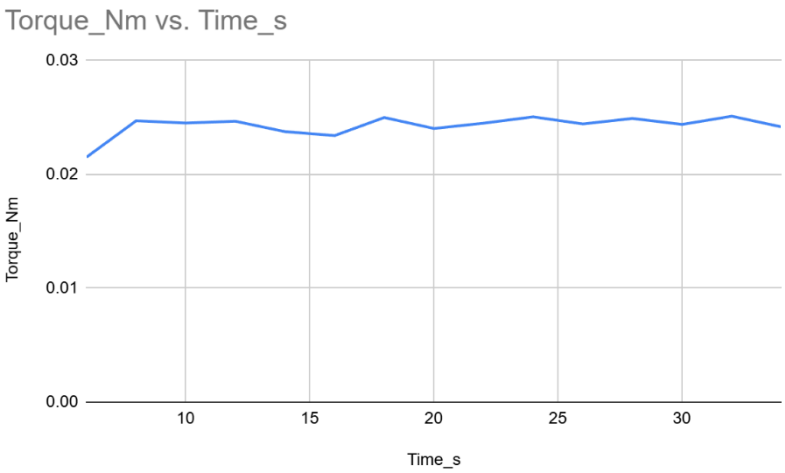


FIGURE 5.2 TORQUE OVER TIME (29.1 PA.S SILICON OIL)

This plot shows that the torque measurement of sample 29.1 Pa.s of silicon oil is stable. However, during testing, certain variations were identified. For high-viscosity samples such as 58.2 Pa·s, some fluctuations were observed in some tests conducted at higher immersion depths (e.g., 0.08 m) and different RPMs. Similar behaviour was occasionally observed in low-viscosity samples at certain test conditions. The measurements obtained under the operational condition of 60 RPM and 0.06 m depth were utilised to calculate errors.

Chapter 6: Conclusion and Future Scope

In this project, a laboratory prototype of a rotational viscometer was developed, designed primarily for measuring the viscosity of fluids relevant to industrial glass applications. The mechanical setup included a rotating rod of 0.024 m diameter and 0.3 m length, operated inside a vessel under the assumption of an infinite cup model. The infinite cup technique served as an alternative to minimise wall effects during measurements because industrial vessels demonstrate variable dimensions.

The measurement accuracy of the system was evaluated through multiple tests using silicone oils of known viscosities. The developed viscometer was able to produce stable and repeatable measurements under most testing conditions. The calculated standard deviation with propagation of uncertainties proved that the system met precise requirements suitable for laboratory work

6.1 Findings

The samples tested included silicone oils of viscosities 9.65 Pa·s, 19.3 Pa·s, 29.1 Pa·s, and 38.8 Pa·s. The mentioned error analysis revealed that the observed viscosities of these samples matched the reference values within acceptable variability ranges.

- For the 9.65 Pa·s sample, a mean viscosity of 9.92 Pa·s was obtained with a standard deviation of 2.88 Pa·s.
- For the 19.3 Pa·s sample, a mean viscosity of 19.36 Pa·s with a standard deviation of 1.35 Pa·s was achieved.
- For the 29.1 Pa·s sample, a mean viscosity of 29.11 Pa·s with a standard deviation of 2.04 Pa·s was recorded.
- For the 38.8 Pa·s sample, a mean viscosity of 38.69 Pa·s with a standard deviation of 1.85 Pa·s was achieved.

Nevertheless, small variations occurred at low immersion depths and high viscosity measurements, but these fluctuations occurred from external mechanical stresses and vibrations instead of sensor or motor internal instability. By selecting the best stable test conditions for error evaluation, a realistic and scientifically reliable estimation of measurement uncertainty was achieved.

Based on the results and error analysis, it was concluded that the system currently operates best within a rotational speed range of 55–85 RPM and an immersion depth range of 0.05–0.09 meters.

Regarding high-temperature fluid measurements, it was concluded that the current prototype is not suitable for direct measurement of molten glass or other high-temperature liquids. The primary limitation arises from the mechanical rod material and other structural components, which are not rated for operation at elevated temperatures. However, the torque sensor and stepper motor components are capable of functioning reliably at ambient temperatures up to 70 °C, allowing the system to be used for moderate-temperature fluid testing.

When compared to the initial objectives, it can be concluded that the development of a functional laboratory prototype for real-time viscosity measurement was successfully achieved. The system proved its accuracy in viscosity measurement, together with its capability to collect data automatically and perform analysis through a Raspberry Pi environment. Results from the experiments proved the fundamental principle while showing that the system serves as a strong base for developing an industrial in-line viscometer in the future.

6.2 Future scope

The development of this laboratory prototype for real-time viscosity measurement was successfully achieved within a limited range of parameters. Firstly, mechanical misalignments between the motor shaft, torque sensor, and rotating rod were found to contribute significantly to measurement errors.

The system performance would improve through new designs that would implement precise couplings as well as enhanced mechanical stability to decrease both vibration and torsional movement and misalignment effects. The stability of measurements will improve significantly when using this method at elevated viscosities and deep immersion conditions. Secondly, the prototype was observed to perform optimally within a rotational speed range of 55–85 RPM and an immersion depth range of 0.05–0.09 meters. So, one of the main objectives will be the expansion of rotational speed and immersion depth range. Better motor control methods and calibration strategies will be implemented to expand the range and to increase accuracy.

Thirdly, the current mechanical rod and vessel setup limits the system to room-temperature measurements. To enable viscosity measurements of high-temperature fluids, such as molten glasses or industrial oils, high-temperature-resistant materials should be incorporated for the rotating rod and vessel. Currently, the torque sensor and stepper motor have an ambient temperature up to 70 °C, for working in an environment of high temperature, more than 70 °C, component upgrades and more thermal shielding will be required.

In addition, the signal conditioning circuit, which was currently implemented on a breadboard during prototype development, has been designed on Altium software for future integration. It is planned that a proper printed circuit board (PCB) will be fabricated and implemented to improve electrical stability, reduce noise, and enhance overall system reliability.

Furthermore, the infinite cup model adopted in this project assumed negligible wall effects. For real industrial applications, vessels of finite dimensions will introduce additional hydrodynamic influences. Therefore, future work should include modelling and compensation techniques to account for vessel-wall effects when transferring the prototype into practical environments.

Furthermore, improvements in the data acquisition system and error analysis could increase the accuracy and reliability of measurements. Implementing wireless communication for data logging and control could also be considered to modernise the system for industrial automation.

The prototype created in this project establishes a strong basis to develop further advancements. The system development will result in a solid industrial in-line viscometer through resolving mechanical issues, together with temperature breakdowns and power problems, and operational restrictions.

Bibliography

(n.d.). <https://www.sciencedirect.com/topics/materials-science/glass>.

Affatigato, M. “Borosilicate Glasses.” *Encyclopedia of materials: Technical Ceramics and Glasses* (2016): 1--6. <https://www.sciencedirect.com/science/article/pii/B9780128185421039989>.

Basumatary, Tenison and Chetia, Dipankar and Singh, Hidam Kumarjit and Bezboruah, Tulshi. “Fibre Optic Sensor Based Viscometer to Measure Viscosity of Newtonian Fluids.” *Proceedings of the IEEE International Conference on Innovations in Electrical Engineering and Computational Technologies* (2017).

Clara, Stefan and Antlinger, Hannes and Abdallah, Ahmad and Reichel, Eva and Hilber, Wolfgang and Jakoby, Bernhard. “A Levitating Sphere Viscometer Operating in a Rotational Mode.” *Proceedings of the IEEE Sensors Conference* (2016).

Fakhari, Ahmad and Galindo-Rosales, Francisco J. “Numerical Assessment of the Penetroviscometer Approach for Large, Rapid and Transient Shear Deformations.” (2020). <https://arxiv.org/abs/2007.13619v1>.

Foundation, Raspberry Pi. “Raspberry Pi Connect (Beta) Documentation.” website. 2025. <https://www.raspberrypi.com/documentation/services/connect.html>.

Glass, History of. “Glass History.” (n.d.). <https://www.historyofglass.com/glass-history/>.

GmbH, Phytron. “Programming Manual for MCC Minilog.” Manual. 2018. <https://www.phytron.de/fileadmin/user_upload/produkte/kommunikation_programmierung/pdf/ma-minilog-mcc-de.pdf>.

—. “Stepper Motor Controller MCC-1-32-48-USB-H Datasheet.” Data sheet. 2023. <https://www.phytron.eu>.

—. “ZSH Stepper Motor Datasheet.” 2023. <https://www.phytron.eu>.

Green, J. *Glass*. 2022. <https://www.sciencedirect.com/science/article/pii/B9780128185421039989>.

Krishnakumar, Ramakrishnan. “Determination of the Three Viscometric Fixed Points for Different Glasses and Preliminary Development of an Online Viscometer for the Glass Industry.” Master Thesis. Deggendorf Institute of Technology, 2022.

LotfizadehDehkordi, Babak and Shiller, Paul J. and Doll, Gary L. “Pressure- and Temperature-Dependent Viscosity Measurements of Lubricants With Polymeric Viscosity Modifiers.” *Frontiers in Mechanical Engineering* (2019).

Mauro, John C. and Yue, Yuanzheng and Ellison, Adam J. and Gupta, Prabhat K. and Allan, Douglas C. “Viscosity of glass-forming liquids.” *Proceedings of the National Academy of Sciences* (2010). <https://www.pnas.org/doi/10.1073/pnas.0911705106>.

Mauro, John C. “Gordon Scott Fulcher: Renaissance Man of Glass Science.” *Frontiers in Materials* (2014).

Nascimento, Marcio Luis Ferreira and Aparicio, Cristina. “Viscosity of strong and fragile glass-forming liquids investigated by means of principal component analysis.” *Journal of Physics and Chemistry of Solids* (2007).

Ojovan, Michael I. “Viscous Flow and the Viscosity of Melts and Glasses.” *Physics and Chemistry of Glasses - European Journal of Glass Science and Technology Part B* (2012).

Sensor and Control Co., Ltd. “ZHKY2050 Dynamic Rotary Torque Sensor Datasheet.” Data sheet. 2023. <https://www.sensor-con.com>.

Shamuratov, Jamshidbek and Mustafayev, Oybek and Kadirov, Islom. “New Viscometers for Measuring the Viscosity of Liquids.” *Journal of Engineering* (2024): Article ID 6877306, 8 pages. <https://doi.org/10.1155/2024/6877306>.

Singh, Parul and Sharma, Kanhaya and Puchades, Ivan and Agarwal, Pankaj B. “A Comprehensive Review on MEMS-Based Viscometers.” *Sensors and Actuators A: Physical* (2022). <https://doi.org/10.1016/j.sna.2022.113456>.

Stillinger, Frank H. and Debenedetti, Pablo G. “Excess viscosity and the glass transition.” *Proceedings of the National Academy of Sciences* (2010). <https://www.pnas.org/doi/10.1073/pnas.1016647107>.

Tanu Shree Roy, Humayun Kabir, Md A. Mannan Chowdhury. “Simple Discussion on Stepper Motors for the Development of Electronic Device.” *International Journal of Scientific & Engineering Research* 5.1 (2014): 1089–1096.

Waterman, J. Cornelissen and H. I. “The Viscosity-Temperature Relationship of Liquids.” *Chemical Engineering Science* 10.4 (1959): 258-268.

Wenhao, Zhang. “Influence of Temperature and Concentration on Viscosity of Complex Fluids.” *Journal of Physics: Conference Series*. 2021.

Zanotto, John C. Mauro and Edgar D. “Two Centuries of Glass Research: Historical Trends, Current Status, and Grand Challenges for the Future.” *International Journal of Applied Glass Science* 5 (2014): 313--327. <https://doi.org/10.1111/ijag.12087>.

Appendix:

6.3 Results and other documents

For the full dataset and other documents, including calibration data, samples, and final raw sensor data:

Dataset and docuents

6.4 Python code

Python code of the entire process:

```
import json
import time
import gpiozero
import threading
import math
import pandas as pd
from datetime import datetime
from openpyxl import Workbook
from openpyxl.utils.dataframe import dataframe_to_rows
from matplotlib.figure import Figure
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as FigureCanvas
from PyQt5 import QtWidgets, QtGui, QtCore
from PyQt5.QtCore import QTimer
import serial

time.sleep(10)
# Serial Port Configuration for Motor
MCC_PORT = '/dev/ttyUSB0'
BAUD_RATE = 115200

# Global Variables
pulse_count = 0
speed_pulse_count = 0
time_data = []
torque_data = []
viscosity_data = []
sensor_rpm_data = []
temperature_data = []
direction_data = []
recording = False
motor_running = False
SAMPLE_TIME = 2.0
user_rpm = 0
json_file_name = None
excel_file_name = None
immersion_depth = []  # Default value
rod_radius = 0.012      # Default value
user_temperature = 25.0  # Default value

# GPIO Configuration for Sensor
PWM_GPIO = 27  # Torque signal
SPEED_GPIO = 22  # Speed signal

# GPIO setup for sensor
```

```
pwm_pin = gpiozero.Button(PWM_GPIO)
speed_pin = gpiozero.Button(SPEED_GPIO)

# Attach interrupts to GPIO pins
def pulse_callback():
    global pulse_count
    pulse_count += 1

def speed_pulse_callback():
    global speed_pulse_count
    speed_pulse_count += 1

pwm_pin.when_pressed = pulse_callback
speed_pin.when_pressed = speed_pulse_callback

# Serial Communication
try:
    ser = serial.Serial(MCC_PORT, BAUD_RATE, timeout=2)
    print("Serial port opened successfully.")
except serial.SerialException as e:
    print(f"Error opening serial port: {e}")
    exit(1)

# Motor Helper Functions
def send_command(address, command):
    try:
        full_command = f"\x02{address}{command}\x03"
        ser.write(full_command.encode())
        time.sleep(0.5)
    except Exception as e:
        print(f"Error sending command: {e}")

def control_motor(rpm, direction):
    global motor_running
    motor_running = True
    send_command("0", "XP45S2")  # Full step mode
    frequency = rpm_to_frequency(rpm)
    direction_command = "XL+" if direction == "CW" else "XL-"
    send_command("0", f"XP14S{frequency}")
    send_command("0", direction_command)

def stop_motor():
    global motor_running
    send_command("0", "XS")  # Stop motor
    motor_running = False

def rpm_to_frequency(rpm, steps_per_revolution=400):
    return int((rpm / 60) * steps_per_revolution)


def calculate_torque_and_viscosity(frequency_khz, rpm, h, r):
    if frequency_khz > 20:  # Filter out frequencies above 20 kHz
        return None, None

    raw_torque_Nm = max(0.1 * frequency_khz - 1, 0)  # Avoid negative torque
    torque_Nm = raw_torque_Nm
    d = r * 2
    ang = (2 * math.pi * rpm) / 60

    viscosity_1 = torque_Nm / (math.pi * d ** 2 * h * max(ang / 2, 1e-6))  # Prevent division by zero

    if torque_Nm == 0:
        #viscosity_2 = 0.0
        viscosity = 0.0   # <<< this was missing in your original code
    else:
        viscosity_2 = viscosity_1 * (0.1176 * torque_Nm ** -0.464)

        viscosity = viscosity_2 * (-150334*torque_Nm**4 + 27627*torque_Nm**3 - 1712.5*torque_Nm**2 + 50.227*torque_Nm + 0.4264)
```

```
    return torque_Nm, viscosity

def save_sensor_data(data):
    global json_file_name
    if json_file_name is None:
        timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
        json_file_name = f"sensor_data_{timestamp}.json"
    try:
        with open(json_file_name, "r") as f:
            existing_data = json.load(f)
    except FileNotFoundError:
        existing_data = []
    existing_data.append(data)
    with open(json_file_name, "w") as f:
        json.dump(existing_data, f, indent=4)

def export_to_excel():
    global excel_file_name
    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    excel_file_name = f"sensor_data_{timestamp}.xlsx"

    data = {
        "Time_s": time_data,
        "Torque_Nm": torque_data,
        "Viscosity_Pa_s": viscosity_data,
        "User_RPM": [user_rpm] * len(time_data),
        #"Sensor_RPM": sensor_rpm_data,
        "Temperature_C": temperature_data,
        "Direction": direction_data,
        "Immersion depth":immersion_depth
    }
    df = pd.DataFrame(data)

    with pd.ExcelWriter(excel_file_name, engine="openpyxl") as writer:
        df.to_excel(writer, index=False, sheet_name="Sensor Data")

    # Save plots
    save_plots(df)

def save_plots(df):
    # Viscosity vs. Time
    fig1 = Figure(figsize=(8, 6))
    ax1 = fig1.add_subplot(111)
    ax1.plot(
        df["Time_s"],
        df["Viscosity_Pa_s"],
        label="Viscosity (Pa.s)",
        marker="o",  # Markers for detail
        linestyle="-"
    )
    ax1.set_xlabel("Time (s)")
    ax1.set_ylabel("Viscosity (Pa.s)")
    ax1.set_title("Viscosity vs. Time (Detailed)")
    ax1.legend()
    ax1.grid(True)
    fig1.savefig("viscosity_vs_time_detailed.png")  # Save plot

    # Viscosity vs. Torque
    fig2 = Figure(figsize=(8, 6))
    ax2 = fig2.add_subplot(111)
    ax2.plot(
        df["Torque_Nm"],
        df["Viscosity_Pa_s"],
        label="Viscosity (Pa.s)",
        marker="o",  # Markers for detail
        linestyle="-"
    )
    ax2.set_xlabel("Torque (Nm)")
```

```
    ax2.set_ylabel("Viscosity (Pa.s)")
    ax2.set_title("Viscosity vs. Torque (Detailed)")
    ax2.legend()
    ax2.grid(True)
    fig2.savefig("viscosity_vs_torque_detailed.png")  # Save plot
# Sensor Data Collection
def collect_sensor_data():
    global pulse_count, speed_pulse_count, time_data, torque_data, viscosity_data, sensor_rpm_data, temperature_data, direction_data,
recording, user_rpm, immersion_depth, rod_radius, user_temperature
    start_time = time.time()
    previous_time = start_time

    while recording:
        current_time = time.time()
        elapsed_time = current_time - previous_time

        if elapsed_time < SAMPLE_TIME:
            continue

        # Calculate sensor RPM and other parameters
        sensor_rpm = speed_pulse_count / elapsed_time
        frequency_khz = (pulse_count / elapsed_time) / 1000.0
        torque, viscosity = calculate_torque_and_viscosity(frequency_khz, user_rpm, immersion_depth, rod_radius)

        if torque is None or viscosity is None:
            pulse_count = 0
            speed_pulse_count = 0
            previous_time = current_time
            continue

        # Store data
        time_data.append(current_time - start_time)
        torque_data.append(torque)
        viscosity_data.append(viscosity)
        sensor_rpm_data.append(sensor_rpm)
        temperature_data.append(user_temperature)

        # Access the direction safely from the global value
        direction = "Clockwise" if window.direction.currentText() == "CW" else "Counterclockwise"
        direction_data.append(direction)

        # Print results to the terminal
        print(f"Time: {current_time - start_time:.2f}s, Frequency: {round(frequency_khz, 3)} kHz, Sensor RPM: {sensor_rpm:.2f}, Torque:
{torque:.3f} Nm, Viscosity: {viscosity:.3f} Pa.s")

        # Reset counters
        pulse_count = 0
        speed_pulse_count = 0
        previous_time = current_time
        time.sleep(SAMPLE_TIME)


# GUI Class
class MotorSensorApp(QtWidgets.QWidget):
    def __init__(self):
        super().__init__()
        self.figure = Figure(figsize=(6, 5))
        self.canvas = FigureCanvas(self.figure)
        self.ax = self.figure.add_subplot(111)
        self.timer = QTimer()
        self.timer.timeout.connect(self.update_plot)
        self.initUI()

    def initUI(self):
        self.setWindowTitle("Rotational Viscometer")
        self.setGeometry(100, 100, 1200, 600)

        # Main Layout
```

```
main_layout = QtWidgets.QVBoxLayout()

# Header
header = QtWidgets.QLabel("ROTATIONAL VISCOMETER")
header.setFont(QtGui.QFont("Arial", 16, QtGui.QFont.Bold))
header.setAlignment(QtCore.Qt.AlignCenter)

sub_header = QtWidgets.QLabel("Technologie Anwender Zentrum")
sub_header.setFont(QtGui.QFont("Arial", 10))
sub_header.setAlignment(QtCore.Qt.AlignRight)

# Controls Panel
controls_layout = QtWidgets.QGridLayout()

self.direction = QtWidgets.QComboBox()
self.direction.addItems(["CW", "CCW"])
self.rpm_input = QtWidgets.QLineEdit()
self.rpm_input.setPlaceholderText("Enter Motor RPM")
self.sample_time_input = QtWidgets.QLineEdit()
self.sample_time_input.setPlaceholderText(f"Sample Time (s) [Default: {SAMPLE_TIME}]")
self.depth_input = QtWidgets.QLineEdit()
self.depth_input.setPlaceholderText(f"Immersion Depth (m) [Default: {immersion_depth}]")
self.radius_input = QtWidgets.QLineEdit()
self.radius_input.setPlaceholderText(f"Rod Radius (m) [Default: {rod_radius}]")
self.temperature_input = QtWidgets.QLineEdit()
self.temperature_input.setPlaceholderText("Enter Temperature (C)")

self.start_motor_btn = QtWidgets.QPushButton("Start Motor")
self.stop_motor_btn = QtWidgets.QPushButton("Stop Motor")
self.start_recording_btn = QtWidgets.QPushButton("Start Recording")
self.stop_recording_btn = QtWidgets.QPushButton("Stop Recording")
self.export_btn = QtWidgets.QPushButton("Export Data")
self.reset_btn = QtWidgets.QPushButton("Reset GUI")

self.motor_status = QtWidgets.QLabel()
self.motor_status.setAlignment(QtCore.Qt.AlignCenter)
self.motor_status.setFont(QtGui.QFont("Arial", 12))
self.update_motor_status(False)

controls_layout.addWidget(QtWidgets.QLabel("Direction:"), 0, 0)
controls_layout.addWidget(self.direction, 0, 1)
controls_layout.addWidget(QtWidgets.QLabel("RPM:"), 1, 0)
controls_layout.addWidget(self.rpm_input, 1, 1)
controls_layout.addWidget(QtWidgets.QLabel("Sample Time:"), 2, 0)
controls_layout.addWidget(self.sample_time_input, 2, 1)
controls_layout.addWidget(QtWidgets.QLabel("Immersion Depth:"), 3, 0)
controls_layout.addWidget(self.depth_input, 3, 1)
controls_layout.addWidget(QtWidgets.QLabel("Rod Radius:"), 4, 0)
controls_layout.addWidget(self.radius_input, 4, 1)
controls_layout.addWidget(QtWidgets.QLabel("Temperature (C):"), 5, 0)
controls_layout.addWidget(self.temperature_input, 5, 1)
controls_layout.addWidget(self.start_motor_btn, 6, 0)
controls_layout.addWidget(self.stop_motor_btn, 6, 1)
controls_layout.addWidget(self.start_recording_btn, 7, 0)
controls_layout.addWidget(self.stop_recording_btn, 7, 1)
controls_layout.addWidget(self.export_btn, 8, 0, 1, 2)
controls_layout.addWidget(self.reset_btn, 9, 0, 1, 2)
controls_layout.addWidget(QtWidgets.QLabel("Motor Status:"), 10, 0)
controls_layout.addWidget(self.motor_status, 10, 1)

# Live Data Panel
data_layout = QtWidgets.QVBoxLayout()
self.result_display = QtWidgets.QTextEdit()
self.result_display.setReadOnly(True)
data_layout.addWidget(self.result_display)

# Plot Panel
plot_layout = QtWidgets.QVBoxLayout()
```

```
        plot_layout.addWidget(self.canvas)

        # Combine Layouts
        top_layout = QtWidgets.QVBoxLayout()
        top_layout.addWidget(header)
        top_layout.addWidget(sub_header)

        bottom_layout = QtWidgets.QHBoxLayout()
        bottom_layout.addLayout(controls_layout)
        bottom_layout.addLayout(data_layout)
        bottom_layout.addLayout(plot_layout)

        main_layout.addLayout(top_layout)
        main_layout.addLayout(bottom_layout)

        self.start_motor_btn.clicked.connect(self.start_motor)
        self.stop_motor_btn.clicked.connect(self.stop_motor)
        self.start_recording_btn.clicked.connect(self.start_recording)
        self.stop_recording_btn.clicked.connect(self.stop_recording)
        self.export_btn.clicked.connect(export_to_excel)
        self.reset_btn.clicked.connect(self.reset_gui)

        self.setLayout(main_layout)

    def start_motor(self):
        global user_rpm, motor_running
        user_rpm = int(self.rpm_input.text())
        direction = self.direction.currentText()
        threading.Thread(target=control_motor, args=(user_rpm, direction), daemon=True).start()
        motor_running = True
        self.update_motor_status(True)

    def stop_motor(self):
        stop_motor()
        self.update_motor_status(False)

    def start_recording(self):
        global recording, SAMPLE_TIME, immersion_depth, rod_radius, user_temperature
        try:
            if self.sample_time_input.text():
                SAMPLE_TIME = float(self.sample_time_input.text())
            if self.depth_input.text():
                immersion_depth = float(self.depth_input.text())
            if self.radius_input.text():
                rod_radius = float(self.radius_input.text())
            if self.temperature_input.text():
                user_temperature = float(self.temperature_input.text())
        except ValueError:
            print("Invalid input. Using default values.")

        recording = True
        threading.Thread(target=collect_sensor_data, daemon=True).start()
        self.timer.start(1000)

    def stop_recording(self):
        global recording
        recording = False
        self.timer.stop()

    def reset_gui(self):
        global time_data, torque_data, viscosity_data, sensor_rpm_data, temperature_data, direction_data
        time_data.clear()
        torque_data.clear()
        viscosity_data.clear()
        sensor_rpm_data.clear()
        temperature_data.clear()
        direction_data.clear()
        self.result_display.clear()
```

```
        self.ax.clear()
        self.canvas.draw()
        self.update_motor_status(False)
        print("GUI Reset Complete.")

    def update_motor_status(self, running):
        if running:
            self.motor_status.setText("Running")
            self.motor_status.setStyleSheet("color: green;")
        else:
            self.motor_status.setText("Stopped")
            self.motor_status.setStyleSheet("color: red;")

    def update_plot(self):
        self.ax.clear()
        self.ax.plot(
            time_data,
            viscosity_data,
            label="Viscosity (Pa.s)",
            marker="o",  # Add markers for better detail
            linestyle="-"
        )
        self.ax.set_xlabel("Time (s)")
        self.ax.set_ylabel("Viscosity (Pa.s)")
        self.ax.set_title("Viscosity vs. Time (Detailed)")
        self.ax.legend()
        self.ax.grid(True)  # Enable grid for clarity
        self.canvas.draw()
        # Append the latest results to the results display
        if torque_data and viscosity_data:
            self.result_display.append(f"Time: {time_data[-1]:.2f}s, Torque: {torque_data[-1]:.3f} Nm, Viscosity: {viscosity_data[-1]:.3f} Pa.s")
window = None
# Run Application
app = QtWidgets.QApplication([])
window = MotorSensorApp()  # Set the global window reference here
window.show()
app.exec_()

# Close Serial Port
ser.close()
```